CALIDAD **5**
¿Qué es la calidad? 5
¿Qué cosas ocurren frecuentemente en los proyectos de desarrollo de software? 5
Principios 6
¿A quién tiene que satisfacer? 6
Las cinco visiones de la calidad 7
El diagrama de las tres calidades 7

Calidad de Producto **8**
Modelo 1: ISO/IEC 25000 (familia SQuaRE) 8
Modelo 2: Barbacci / SEI 9
Modelo 3: McCall 9

Calidad y Proceso de Desarrollo **10**
Definición de proceso 10
¿Cómo lo definimos? 11
¿Cómo es un proceso para construir software? 11

Aseguramiento de Calidad de Software **12**
¿Qué es la administración de la calidad del software? 12
El Grupo de Aseguramiento de Calidad (GAC): ¿a quién reporta? 12
Las tres actividades de la administración de calidad 12
Funciones del Aseguramiento de Calidad de Software 13
La Administración de Calidad y el Desarrollo de Software 13
Procesos basados en calidad 13
Calidad de Procesos en la Práctica 14
Estándares y Aseguramiento de Calidad 14
Planificación de la Calidad 15
Control de Calidad 15
Revisiones de Calidad 15

Modelos de Mejora **16**
SPICE 16
Modelo IDEAL 16

Modelos, Estándares y Normas de Calidad **17**
Modelo de Calidad CMMI 17
Constelaciones 17
Roles – Grupos 18
Representación por Etapas: niveles 18
Áreas de Proceso por Nivel 18
CCMI vs ágil 19
Conclusión 21

TESTING DE SOFTWARE **21**
¿Qué es el testing? 21
Testing en el contexto: Asegurar la calidad vs. Controlar la calidad 22

Error vs. Defecto **22**

Conceptos: Defectos, severidad y prioridad **23**

Niveles de Prueba **24**
Testing Unitario 24
Testing de Integración 24
Testing de Sistema 25
Testing de Aceptación 25

Los 4 ambientes ideales **26**
¿Por qué se separan en varios ambientes? 26

Caso de Prueba **27**
Derivación de casos de prueba 27

Condiciones de prueba **27**

Métodos **27**
Caja Negra 28
Caja Blanca 28

Ciclo de Prueba **28**

Regresión **29**

Proceso de Pruebas **29**
1 Planificación de pruebas → Plan de prueba 29
2 Diseño de casos de prueba → Casos de prueba 30
3 Ejecución 30
4 Reporte de defectos → Reporte de defectos 30
Por qué ejecución y reporte son recursivos 31

Verificación y Validación **31**

¿Cuánto testing es suficiente? **31**

Principios del Testing **32**

Tipos de pruebas **32**
Testing funcional 32
Testing no funcional 33

TESTING ÁGIL **34**

Manifesto del Testing **34**

Ciclo de vida del testing vs. Ágil **35**

la cascada vs. el ciclo iterativo **35**

las actividades dentro de una iteración **35**

9 principios del testing ágil **36**

6 Prácticas Concretas **37**

El rol del Tester en los equipos ágiles **38**

Pirámide del Testing Tradicional vs. Agile **38**

¿Qué es automatizar el testing? **39**
Por qué es necesaria en ágil 39
Beneficios de la automatización 40

LEAN Y KANBAN **40**

Filosofía Lean **40**
Principios 40
7 + 1 Desperdicios 42

KANBAN **43**
Valores de Kanban 44
Principios de Kanban 45
Principios de gestión del cambio 45
Principios de entrega de servicios 45
Prácticas Generales de KANBAN 45

MÉTRICAS **46**

Métricas en el enfoque tradicional (basado en procesos definidos) **46**
Métricas de software 46
Métricas básicas para un proyecto de software 47

Métricas en ambientes ágiles (para procesos empíricos) **47**
Dos principios ágiles que guían la elección de las métricas 47
Velocidad 48
Capacidad 48
Running Tested Features (RTF) 48

Métricas en Kanban (procesos empíricos con enfoque Lean) **48**
Kanban: métricas clave 48
Cycle Time (tiempo de ciclo) 49
Lead Time (tiempo de entrega) 49
Touch Time (tiempo de tocado) 49
Métricas orientadas a servicio 49

Resumiendo métricas en cada enfoque **50**

¿Y para el producto software, qué métricas? **50**
Tamaño 50
Defectos 50

¿Cuál es mejor? **50**

AUDITORÍAS **51**

Diferencia con una revisión técnica **51**

Tipos **51**

Los pasos de la auditoría: **51**

Hallazgos **52**

Auditoría al grupo de calidad. **52**

Checklist de auditoría **52**

REVISIONES TÉCNICAS **53**

Objetivo **53**

Verificación y Validación **53**

Falla y producto de trabajo **53**

Ventajas **53**

Desventajas **53**

Tipos **53**
Inspección 54
Walkthrough (recorrida) 54

Calidad

¿Qué es la calidad?

Hay varias formas de definirla:

- Calidad son todos los aspectos y características de un producto o servicio que se relacionan con su habilidad de alcanzar las necesidades **manifiestas o implícitas**. Fijate en la palabra "implícitas": no alcanza con cumplir lo que el cliente pidió explícitamente. También cuenta lo que da por sentado sin decirlo, por ejemplo que el sistema no pierda datos o que responda rápido.
- Un proceso eficaz de software que se aplica de manera que crea un producto útil que proporciona valor medible a quienes lo producen y a quienes lo utilizan. Acá aparecen tres ideas: el **proceso** importa, no solo el producto final; el producto tiene que ser **útil**; y el valor es para **ambos lados**, quien lo usa y quien lo hace.
- En teoría, calidad sería "cumplir la especificación". En software eso no alcanza por tres motivos: las especificaciones suelen estar incompletas, los requerimientos no funcionales son difíciles de especificar, y cliente y desarrollador pueden buscar cosas distintas. Por eso concluye que la calidad del software es en parte **subjetiva**.

¿Qué cosas ocurren frecuentemente en los proyectos de desarrollo de software?

- Atrasos en las entregas: el proyecto no cumple las fechas comprometidas.
- Costos excedidos: se gasta más de lo presupuestado, casi siempre por retrabajo.
- Falta de cumplimiento de los compromisos: con el cliente y dentro del equipo.
- Requerimientos poco claros: si no se sabe bien qué construir, se construye mal o se rehace. Es una de las causas de fondo de casi todo lo demás.
- El software no hace lo que tiene que hacer: falla la validación ("¿construimos el producto correcto?").
- Trabajo fuera de hora: consecuencia directa de los atrasos. El equipo compensa con horas extra, se cansa y comete más errores. Es un círculo vicioso.
- Fenómeno del 90-90: el primer 90% del código lleva el 90% del tiempo, y el 10% restante lleva otro 90%. Es decir, se cree que el proyecto está casi terminado, pero lo que falta (integrar, corregir, pulir) lleva tanto como todo lo anterior.
- "¿Dónde está ese componente?": no se sabe cuál es la versión correcta de un archivo o dónde quedó algo. Es síntoma de que falta gestión de configuración.

Principios

1. **La calidad no se "inyecta" ni se compra; debe estar embebida.** No se le agrega calidad al producto al final, por ejemplo testeando más. Tiene que construirse en cada actividad del proceso. Tampoco se resuelve comprando una herramienta o una certificación.
2. **Es un esfuerzo de todos.** No es tarea de un tester ni de un área de calidad. Cada persona que produce algo es responsable de su calidad. Este principio vuelve en testing ágil: "todo el equipo es responsable".
3. **Las personas son la clave para lograrlo.** Procesos y herramientas ayudan, pero quien hace el trabajo es la gente.
4. **Capacitación.** Consecuencia del anterior: si las personas son la clave, hay que formarlas en el proceso, las técnicas y los estándares.
5. **Se necesita sponsor a nivel gerencial.** Sin apoyo de la dirección (recursos, tiempo, prioridad), las iniciativas de calidad mueren.
6. **Pero se puede empezar por uno.** Equilibra al anterior: no hace falta esperar a que toda la organización cambie. Uno puede aplicar buenas prácticas en su propio trabajo.
7. **Se debe liderar con el ejemplo.** Si el líder pide calidad pero él mismo se saltea las revisiones, nadie lo va a tomar en serio.
8. **No se puede controlar lo que no se mide.** Para saber si la calidad mejora o empeora hace falta medir: defectos, tiempos, esfuerzo. Es la base de los niveles altos de CMMI.
9. **Simplicidad: empezar por lo básico.** No implementar un proceso enorme de golpe. Arrancar con prácticas simples y crecer.
10. **El aseguramiento de la calidad debe planificarse.** No se improvisa. Por eso existe el plan de calidad y la planificación es una de las actividades de la administración de calidad.
11. **El aumento de las pruebas no aumenta la calidad.** Este es el más preguntado. El testing **detecta** defectos, no los **previene**. Probar más encuentra más problemas, pero el producto no fue construido mejor. Se conecta con la frase "el testing no puede asegurar la calidad".
12. **Debe ser razonable para mi negocio.** El nivel de calidad y el esfuerzo invertido dependen del contexto. No se le exige lo mismo a una app de recetas que al software de un marcapasos. Pressman lo trata como software "suficientemente bueno".

¿A quién tiene que satisfacer?

Un software de calidad satisface:

- Las expectativas del cliente (quien paga y pide el sistema);
- Las expectativas del usuario (quien lo usa todos los días, que muchas veces no es el cliente);
- Las necesidades de la gerencia (costos, plazos, rentabilidad);
- Las necesidades del equipo de desarrollo y mantenimiento (que sea entendible, modificable y probable);
- Las de otros interesados.

Las cinco visiones de la calidad

Visión	Qué considera calidad	Imagen de la diapositiva
Trascendental	Algo que se reconoce de inmediato pero no se puede definir con precisión ("lo veo y sé que es bueno")	La del centro: abarca a todas
Del usuario	Que el producto cumpla las metas concretas del usuario final	El usuario con la computadora en llamas
De manufactura	Que el producto se ajuste a la especificación original	Obreros en la línea de producción
Del producto	Las características inherentes y medibles del producto (funciones, atributos)	La computadora con muchos brazos
Basada en el valor	Cuánto está dispuesto a pagar el cliente por el producto	El de la bolsa de plata

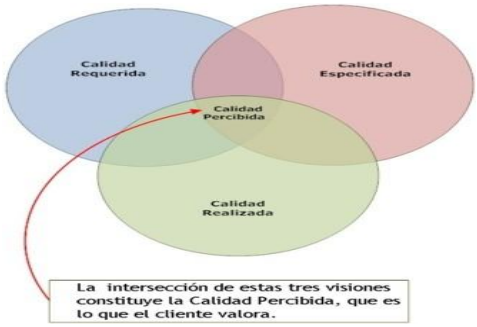


Ninguna visión alcanza sola. Un producto puede cumplir su especificación al 100% (visión de manufactura) y aun así no servirle al usuario.

El diagrama de las tres calidades

- **Calidad necesaria:** lo que el cliente realmente necesita.
- **Calidad programada:** lo que se planificó y especificó construir.
- **Calidad realizada:** lo que efectivamente se construyó.

Sin gestión, los tres círculos se superponen poco. Con **gestión de la calidad**, se busca que coincidan lo más posible, y esa zona común es la **calidad esperada**.



Calidad de Producto

"Calidad de Producto" responde a dos preguntas: **cómo se mide** la calidad de un producto de software (con modelos) y **con qué técnicas se asegura y se controla**.

La calidad de producto mira el software en sí: qué tan bien hace lo que tiene que hacer, qué tan fácil es usarlo, mantenerlo o instalarlo. El problema es que "bueno" es abstracto. Para poder medirlo, los modelos de calidad descomponen la calidad en **características**, cada característica en **subcaracterísticas**, y estas en **métricas** concretas.



Modelo 1: ISO/IEC 25000 (familia SQuaRE)

Agrupa tres modelos:

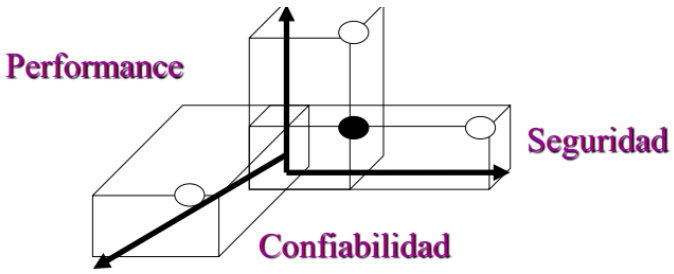
- **Calidad de producto** (ISO 25010:2011): las características del software en sí.
- **Calidad en uso** (ISO 25010:2011): lo que experimenta el usuario al usarlo en su contexto real (efectividad, eficiencia, satisfacción, ausencia de riesgos, cobertura del contexto).
- **Calidad de datos** (ISO 25012:2008): exactitud, completitud y consistencia de los datos.

El que más importa es el árbol de **calidad de producto de ISO 25010**, con **8 características**:

Característica	Pregunta que responde	Subcaracterísticas
Adecuación funcional	¿Hace lo que tiene que hacer?	Completitud, corrección, pertinencia funcional
Eficiencia de desempeño	¿Qué tan rápido y con cuántos recursos?	Comportamiento temporal, utilización de recursos, capacidad
Compatibilidad	¿Convive y se comunica con otros sistemas?	Coexistencia, interoperabilidad
Usabilidad	¿Es fácil de usar?	Reconocimiento de adecuación, aprendizaje, operabilidad, protección frente a errores de usuario, estética, accesibilidad
Fiabilidad	¿Funciona sin fallar?	Madurez, disponibilidad, tolerancia a fallos, capacidad de recuperación
Seguridad	¿Protege la información?	Confidencialidad, integridad, no repudio, autenticidad, responsabilidad
Mantenibilidad	¿Es fácil de modificar?	Modularidad, reusabilidad, analizabilidad, capacidad de ser modificado y de ser probado
Portabilidad	¿Se puede llevar a otro entorno?	Adaptabilidad, facilidad de instalación, capacidad de ser reemplazado

Modelo 2: Barbacci / SEI

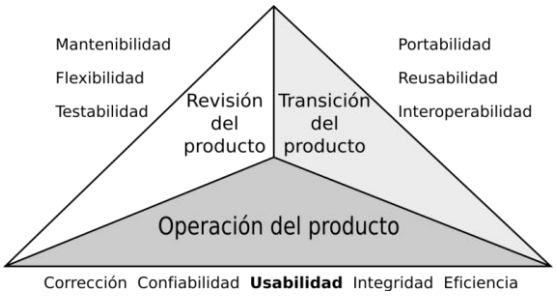
Se centra en pocos atributos (**performance, seguridad y confiabilidad**) y plantea una idea clave: optimizar un atributo al máximo es un **óptimo local**, y suele empeorar los otros. Por ejemplo, más seguridad (cifrado, validaciones) puede bajar la performance. Lo que se busca es un **óptimo global**: el mejor balance entre varios atributos a la vez.



Modelo 3: McCall

Es el modelo clásico (1977). Organiza los factores de calidad en tres perspectivas, como los lados de un triángulo:

- **Operación del producto** (cómo funciona en el uso diario): corrección, fiabilidad, usabilidad, integridad, eficiencia.
- **Revisión del producto** (qué tan fácil es cambiarlo): facilidad de mantenimiento, flexibilidad, facilidad de prueba.
- **Transición del producto** (qué tan fácil es llevarlo a otro entorno): portabilidad, reusabilidad, interoperabilidad.

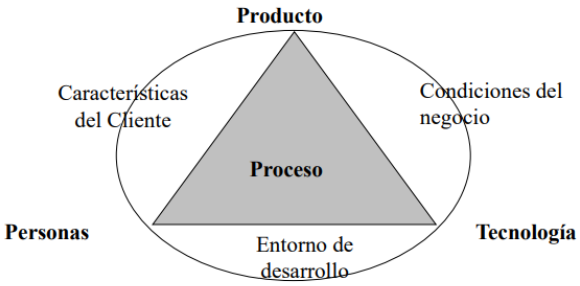


Calidad y Proceso de Desarrollo

No existe un proceso único que sirva para todos los proyectos. Hay que adaptarlo al contexto (tamaño del equipo, tipo de producto, cliente, criticidad). Conecta con el principio "debe ser razonable para mi negocio".

"El proceso es el único factor controlable"

Es la frase más importante del bloque: el proceso es el único factor "controlable" al mejorar la calidad del software y su rendimiento como organización.



Alrededor, en el círculo, están los factores del entorno: **características del cliente**, **condiciones del negocio** y **entorno de desarrollo**.

¿Por qué el proceso es "el único controlable"? Porque los otros factores la organización no los maneja del todo, o no los puede cambiar rápido. No elige cómo es el cliente ni las condiciones del mercado. Las personas rotan, y no se puede garantizar tener siempre a los mejores. La tecnología cambia constantemente. El proceso, en cambio, se puede **definir, medir y mejorar**, y es lo que conecta a personas, tecnología y producto. Por eso los modelos como CMMI se centran en el proceso.

Definición de proceso

- **Proceso (IEEE):** la secuencia de pasos ejecutados para un propósito dado.
- **Proceso de software (SW-CMM):** un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.

Un proceso integra tres elementos:

- **Personas** con habilidades, entrenamiento y motivación.
- **Procedimientos y métodos:** las tareas y cómo se relacionan (A → B → C → D).
- **Herramientas y equipos.**

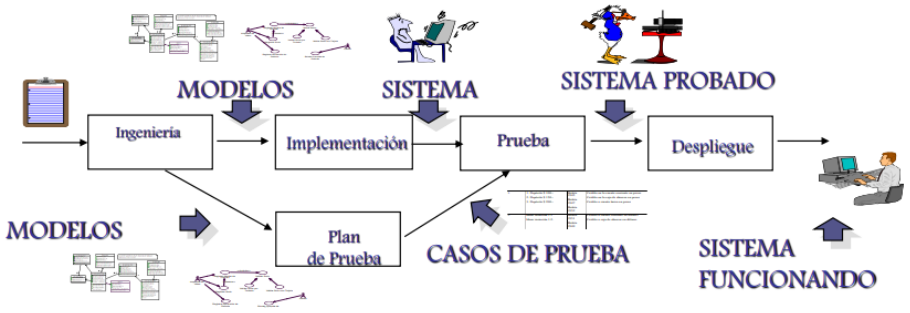
Si falta uno, el proceso no funciona. Por ejemplo, un buen método con gente sin capacitación, o buena gente sin herramientas.

¿Cómo lo definimos?

Se parte de "un proceso sin definir" (una caja negra con una entrada y una salida) y se descompone en **etapas**, y cada etapa en **tareas**, algunas en paralelo y otras en secuencia. Para cada parte hay que responder cuatro preguntas:

- **¿Quiénes** deben participar?
- **¿Qué** deben hacer?
- **¿Cuándo** lo deben hacer?
- **¿Cómo** lo deben hacer?

¿Cómo es un proceso para construir software?



El proceso de construcción de software sigue un flujo principal de cuatro etapas, donde cada una genera un producto que alimenta a la siguiente:

1. **Ingeniería:** a partir de los requerimientos, se elaboran los **modelos** del sistema (por ejemplo, casos de uso y diagramas de diseño).
2. **Implementación:** con esos modelos se construye el **sistema**.
3. **Prueba:** el sistema se verifica y se obtiene un **sistema probado**.
4. **Despliegue:** el sistema se pone en producción y queda un **sistema funcionando** para el usuario.

En paralelo al flujo principal, los modelos de ingeniería también se usan para elaborar el **plan de prueba**, del cual surgen los **casos de prueba** que luego se ejecutan en la etapa de prueba. Esto muestra que las pruebas no se piensan al final, sino que se planifican desde el comienzo, a partir de los mismos modelos con que se construye el sistema.

Además, existen tres actividades que **atraviesan todo el proceso**, de principio a fin:

- **Planificación y seguimiento de proyectos:** planificar el trabajo y controlar su avance.
- **Administración de configuraciones:** controlar las versiones y los cambios de todos los productos que se generan.
- **Aseguramiento de la calidad:** verificar que se cumplan los procesos y estándares definidos.

Que el aseguramiento de la calidad sea una actividad transversal, y no una etapa más, refleja que la calidad no se agrega al final del desarrollo. Debe estar presente en cada una de las etapas del proceso.

Aseguramiento de Calidad de Software

"Lo que no está controlado no está hecho" (Watts Humphrey, uno de los creadores del CMM). Significa que no alcanza con decir que algo se hizo: tiene que haber evidencia de que se hizo y de que se hizo bien.

¿Qué es la administración de la calidad del software?

- **Asegurar** que se alcancen los niveles de calidad requeridos para el producto.
- **Definir** estándares y procesos de calidad apropiados, y **asegurar que se respeten**.
- Desarrollar una **cultura de calidad**, donde la calidad es responsabilidad de todos y cada uno.

El tercer punto conecta con el principio "es un esfuerzo de todos": la administración de calidad no busca que un área controle a las demás, sino que todos trabajen con calidad.

El Grupo de Aseguramiento de Calidad (GAC): ¿a quién reporta?

Tres reglas:

- **No debería reportar al gerente de proyecto.**
- No debería haber más de una posición entre la gerencia de primer nivel y el GAC.
- Cuando sea posible, debería reportar a alguien realmente interesado en la calidad.

La razón de fondo es la **independencia**. El gerente de proyecto debe cumplir presupuesto y calendario. Ante problemas, puede estar tentado de sacrificar la calidad para cumplir los plazos. Por eso el GAC tiene que ser independiente.

Las tres actividades de la administración de calidad

La administración de calidad se divide en tres actividades que funcionan como una cadena, de lo general a lo particular:

- **Aseguramiento de calidad:** sucede a nivel de **toda la organización**. Se definen los estándares y procedimientos que todos los proyectos deberían seguir: por ejemplo, cómo se documentan los requerimientos, cómo se hacen las revisiones de código, cómo se registran los defectos. Es el "manual" de la organización.
- **Planificación de calidad:** sucede al **inicio de cada proyecto**. No todos los proyectos son iguales, así que de ese manual se eligen los estándares que aplican, se adaptan los que hace falta y, si el proyecto lo requiere, se agregan nuevos. El resultado es el plan de calidad del proyecto.
- **Control de calidad:** sucede **durante el proyecto**. Se verifica que el equipo efectivamente esté trabajando según lo planificado.

Un ejemplo para unirlas: la organización define que todo código debe pasar por una revisión de pares (aseguramiento). En un proyecto chico y de bajo riesgo se decide que alcanza con revisar los módulos críticos (planificación). Durante el proyecto, el GAC verifica que esos módulos efectivamente hayan sido revisados (control).

Funciones del Aseguramiento de Calidad de Software

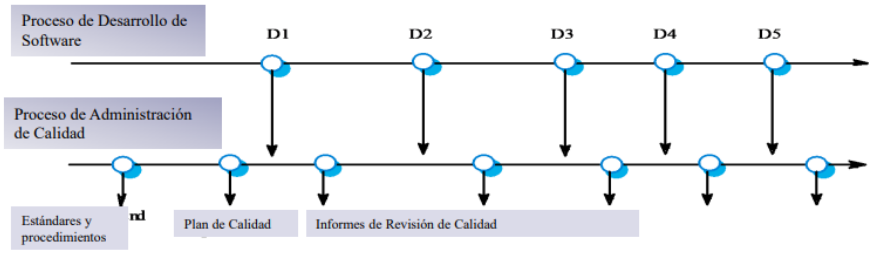
"¿qué hace concretamente el SQA?". La respuesta es que evalúa cada etapa del desarrollo: la planificación del proyecto, los requerimientos, el diseño, la programación, la integración y prueba, y el seguimiento y control del proyecto.

Lo importante es entender qué evalúa. No se trata de que el GAC rehaga el trabajo técnico ni de que revise línea por línea si el diseño es bueno, porque eso lo hacen los ingenieros en las revisiones técnicas. El GAC verifica que cada etapa se haya hecho según el proceso y los estándares definidos: que exista el plan, que se hayan revisado los requerimientos, que se hayan seguido las convenciones de código, que las pruebas se hayan registrado. Pressman lo describe así: el grupo de SQA identifica, documenta y da seguimiento a las desviaciones respecto del proceso, y reporta los incumplimientos a la dirección hasta que se resuelven.

La Administración de Calidad y el Desarrollo de Software

Son dos procesos en paralelo. El de calidad empieza **antes** que el de desarrollo, con los **estándares y procedimientos** y el **plan de calidad**. Después, cada entregable del desarrollo (D1, D2...) pasa por el proceso de calidad, que genera **informes de revisión de calidad**.

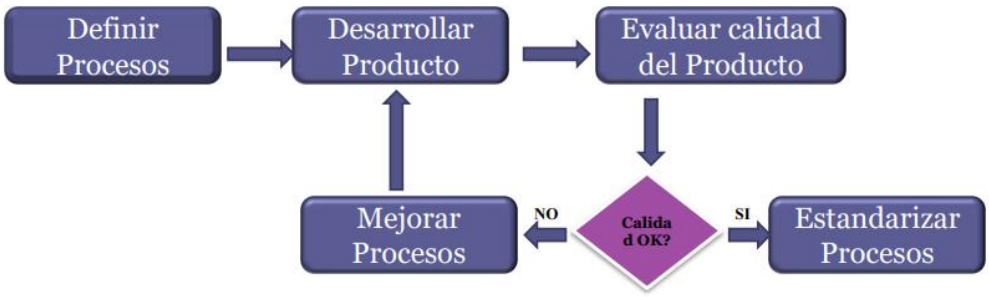
La gestión de calidad brinda una **comprobación independiente** del proceso de desarrollo.



Procesos basados en calidad

La calidad del producto depende de la calidad del proceso que lo produce. Por eso, cuando el producto no alcanza la calidad esperada, no alcanza con corregir ese producto. Hay que corregir el **proceso**, para que los próximos productos salgan bien desde el principio.

La idea viene de la **manufactura**: una vez que las máquinas están bien calibradas, los productos salen bien una y otra vez. En software es más complejo, porque **el software se diseña, no se fabrica**: influyen mucho las habilidades de las personas, la novedad del producto y la presión por los plazos. Aun así, concluye que un buen proceso aumenta mucho la probabilidad de obtener un buen producto.



}

Calidad de Procesos en la Práctica

Primero hay que definir procesos estándar para las actividades clave, como cómo se conduce una revisión o cómo se gestionan las versiones. Sin un proceso definido no hay contra qué comparar.

Después hay que monitorear que esos procesos se cumplan durante el desarrollo, y reportar a la gerencia de proyecto y al responsable del software. El monitoreo sin reporte no sirve, porque nadie toma decisiones con esa información.

El último punto equilibra todo lo anterior: no usar prácticas inapropiadas solo porque están en el estándar. Un estándar es una herramienta, no un fin. Si una práctica no agrega valor en un proyecto concreto, aplicarla igual es burocracia y genera rechazo en el equipo. Por eso la planificación de calidad permite adaptar los estándares a cada proyecto.

Estándares y Aseguramiento de Calidad

Los estándares son la clave de la administración de calidad porque son lo que permite comparar. Como la calidad es en parte subjetiva, sin estándares cada persona tendría su propia idea de qué está “bien hecho”. El estándar fija un criterio común y objetivo: si el código cumple las convenciones, si el documento tiene las secciones requeridas, si la revisión siguió los pasos definidos.

Sommerville agrega otras dos razones. Los estándares guardan la experiencia de la organización: muchas veces surgen de errores pasados, y evitan repetirlos. Además, dan continuidad: si alguien deja el proyecto, quien lo reemplaza encuentra el trabajo hecho de una forma conocida.

- Estándares de producto: se aplican a lo que se construye. Ej.: el formato del documento de requerimientos, el estilo de programación, la plantilla de un caso de prueba.
- Estándares de proceso: se aplican a cómo se trabaja. Ej.: los pasos de una revisión de diseño, el procedimiento para aprobar un cambio, el proceso de liberación de una versión.

Planificación de la Calidad

El plan de calidad es el documento donde cada proyecto define qué significa "calidad" para ese proyecto en particular. Esto es necesario porque no todos los atributos de calidad pesan igual. En un sistema bancario la seguridad es crítica; en un videojuego importan más el desempeño y la usabilidad. Además, mejorar un atributo puede empeorar otro: más seguridad puede significar menos velocidad. Si el plan no lo explicita, cada miembro del equipo va a priorizar según su criterio, y probablemente en direcciones distintas.

Por eso el plan define cuáles son los atributos de calidad más importantes, cómo se va a evaluar cada uno, y qué estándares de la organización se aplican (o si hace falta crear alguno nuevo). Sommerville da un consejo práctico: el plan tiene que ser breve, porque si es muy extenso nadie lo lee y pierde su propósito.

Control de Calidad

El control de calidad es la verificación, durante el proyecto, de que **realmente se está trabajando como se planificó**. Tiene dos enfoques que se complementan:

- **Revisiones de calidad:** las hacen personas que examinan los productos y el proceso. Sirven para lo que requiere juicio, como si un diseño es adecuado o si un documento es claro.
- **Evaluaciones automáticas y mediciones:** las hacen herramientas, como analizadores de código que verifican convenciones o calculan complejidad. Sirven para lo que es repetitivo y objetivo, y tienen la ventaja de ser rápidas y no depender de la atención de una persona.

Revisiones de Calidad

Las revisiones son el **principal método** para validar la calidad, tanto de un proceso como de un producto. Consisten en que un grupo examina una parte del trabajo, junto con su documentación, buscando problemas. Su gran ventaja es que se pueden aplicar desde el comienzo, sobre documentos y modelos, sin necesidad de tener código ejecutable. Así los problemas se encuentran temprano, cuando corregirlos es más barato.

Tres tipos según su objetivo:

- Inspecciones para remoción de defectos: se centran en el producto y buscan errores concretos, por ejemplo en el código o en un diseño.
- Revisiones de progreso: miran producto y proceso para evaluar si el proyecto avanza según lo planificado en tiempo, costo y alcance.
- Revisiones de calidad: comparan el producto con los estándares, para verificar que cumpla lo que la organización definió.

Modelos de Mejora

Una organización puede saber que su proceso tiene problemas: lo detectó en el ciclo de "procesos basados en calidad", o una evaluación CMMI le marcó debilidades. Pero mejorar un proceso no es algo que se haga de un día para otro ni improvisando. Implica cambiar la forma de trabajar de muchas personas, y eso genera resistencia y cuesta tiempo y dinero. Un modelo de mejora da un camino ordenado para hacerlo: por dónde empezar, cómo decidir qué mejorar primero y cómo instalar los cambios para que perduren.

SPICE

Es un modelo para **evaluar la capacidad de los procesos** y, a partir de esa evaluación, orientar su mejora.

SPICE es el nombre con el que se conoce a la norma **ISO/IEC 15504**. Evalúa **cada proceso por separado**, asignándole un nivel de capacidad de 0 a 5. En ese sentido se parece a la representación continua de CMMI, que se ve más adelante. La sigla oficial es Software Process Improvment Capability Evaluation.

Modelo IDEAL

IDEAL es un modelo del **SEI**, el mismo instituto que creó CMMI. Describe cómo llevar adelante un programa de mejora de procesos. Su nombre viene de las iniciales de sus cinco fases en inglés, y es un **ciclo**: cuando termina, vuelve a empezar con la siguiente mejora.

1. **Initiating (Iniciar)**
   Algo dispara la necesidad de mejorar: muchos defectos, un cliente que exige una certificación o proyectos que siempre se atrasan (el *estímulo para la mejora*). En esta fase se define el contexto y los objetivos, se consigue el **patrocinio de la gerencia** y se arma la infraestructura para la mejora.
2. **Diagnosing (Diagnosticar)**
   Se evalúa **cómo se trabaja hoy** y se lo compara con cómo se quiere trabajar. Esa diferencia muestra qué hay que mejorar. Esta evaluación suele hacerse con un modelo de referencia, como CMMI. El resultado son recomendaciones documentadas.
3. **Establishing (Establecer)**
   Como no se puede mejorar todo a la vez, se **fijan prioridades y estrategias**. Se forman equipos de acción para cada mejora y se elabora un plan.
4. **Acting (Actuar)**
   Se ejecuta la mejora. Se definen los procesos nuevos y cómo se van a medir, se prueban primero en **proyectos piloto** y, si funcionan, se instalan en toda la organización, con seguimiento. El piloto es clave: permite probar el cambio a pequeña escala y ajustarlo antes de imponerlo a todos.
5. **Leveraging (Aprender)**
   Se analizan las **lecciones aprendidas**: qué funcionó, qué no y por qué. Con eso se revisa el enfoque para el próximo ciclo de mejora.

Modelos, Estándares y Normas de Calidad

Existe una cantidad enorme de modelos, normas y estándares de calidad y de procesos, y que están relacionados entre sí: unos reemplazan a otros, unos se basan en otros.

Modelo de Calidad CMMI

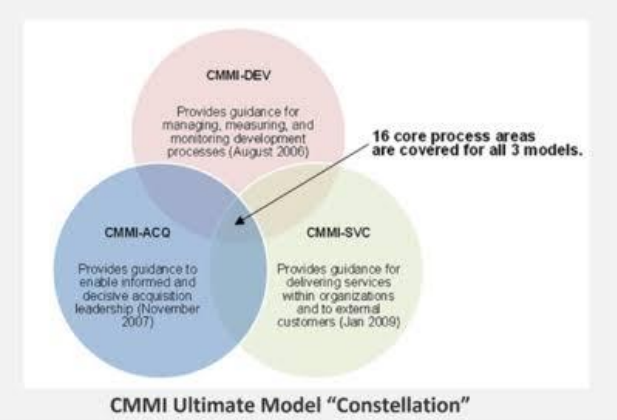
CMMI significa Capability Maturity Model Integration, es decir, modelo integrado de madurez y capacidad.

- Es **uno de los modelos más implementados** del mundo.
- **No es una norma y no se "certifica".** Esto es muy preguntado. A diferencia de ISO 9001, una organización no obtiene un "certificado CMMI". Lo que se hace es una **evaluación** (appraisal) a cargo de profesionales reconocidos por el SEI, llamados **Lead Appraisers**. El resultado indica en qué nivel está la organización según el modelo.

Constelaciones

CMMI no es un único modelo, sino una familia de modelos llamados **constelaciones**, cada uno pensado para un tipo de organización:

- **CMMI-DEV** (*Development*): para organizaciones que **desarrollan** productos. Da la guía para medir, monitorear y administrar los procesos de desarrollo. Es el que estudia la cátedra.
- **CMMI-SVC** (*Services*): para organizaciones que **brindan servicios**, internos o externos. Por ejemplo, una empresa de soporte técnico o de infraestructura.
- **CMMI-ACQ** (*Acquisition*): para organizaciones que **compran** o adquieren productos y servicios. Les da una guía para seleccionar proveedores y administrar esas adquisiciones.



CMMI Ultimate Model "Constellation"

Las tres constelaciones comparten **16 áreas de proceso fundacionales**, comunes a todas, como planificación de proyectos o gestión de configuración. El dato del 77% significa que gran parte de lo que una organización invierte en implementar CMMI-DEV le sirve si después quiere implementar otra constelación.

Roles – Grupos

CMMI menciona roles y grupos, como un grupo de aseguramiento de calidad o un grupo de procesos. Eso puede hacer pensar que una empresa chica no puede aplicar el modelo porque no tiene gente para armar todos esos grupos. Lo importante es que exista alguien responsable de cubrir las actividades de cada rol o grupo. La forma de hacerlo depende del tamaño de la organización. En una empresa grande puede haber un departamento entero de calidad. En una chica, una sola persona puede cubrir varios roles, siempre que las actividades efectivamente se hagan.

Representación por Etapas: niveles

Tiene **5 niveles, del 1 al 5**. Cada nivel se define por un **conjunto de áreas de proceso** que hay que cumplir. Los niveles indican la **madurez de la organización** como un todo.

Los cinco niveles de madurez se representan como una **escalera**, porque se suben de a uno: no se puede saltar del 1 al 3.

1. **Inicial:** los procesos son **impredecibles**, están poco controlados y la organización es **reactiva** (apaga incendios). El éxito depende del esfuerzo heroico de algunas personas, no de la organización. Es el "proceso sin definir" del laberinto.
2. **Administrado:** los procesos están **caracterizados por proyecto**: cada proyecto planifica, hace seguimiento y controla su trabajo. Pero la organización todavía es a menudo **reactiva**, y cada proyecto puede trabajar distinto.
3. **Definido:** los procesos están **caracterizados por la organización**: hay un proceso estándar que todos los proyectos adaptan. La organización pasa a ser **proactiva**: anticipa los problemas en lugar de reaccionar.
4. **Cuantitativamente administrado:** los procesos se **miden y controlan con datos** (estadísticas, métricas). Se pueden hacer predicciones basadas en números.
5. **Optimizado:** el **foco está en la mejora continua** de los procesos. Es el ciclo de "procesos basados en calidad" funcionando de forma sistemática en toda la organización.

A medida que se sube de nivel, aumentan la productividad y la calidad, y disminuye el riesgo. La diferencia que más se confunde es entre el nivel 2 y el 3. En el 2, cada proyecto tiene su proceso. En el 3, la organización tiene un proceso común que los proyectos adaptan.

Áreas de Proceso por Nivel

Un área de proceso es un conjunto de prácticas relacionadas que, aplicadas juntas, cumplen un objetivo importante para mejorar en ese aspecto. Por ejemplo, el área "Gestión de Configuración" agrupa las prácticas para controlar versiones y cambios.

Para alcanzar un nivel de madurez, la organización tiene que cumplir las áreas de proceso de ese nivel **y de todos los anteriores**:

- **Nivel 2, Administrado:** áreas para gestionar **proyectos individuales**. Administración de Requerimientos (REQM), Planificación de Proyectos (PP), Monitoreo y Control de Proyectos (PMC), Administración de Acuerdos con Proveedores (SAM), Medición y Análisis (MA), **Aseguramiento de Calidad de Proceso y Producto (PPQA)** y Administración de Configuración (CM).
- **Nivel 3, Definido:** se suman las áreas de **ingeniería** y las que **estandarizan a nivel organización**. Desarrollo de Requerimientos (RD), Solución Técnica (TS), Integración de Producto (PI), Verificación (VER), Validación (VAL), Foco en el Proceso Organizacional (OPF), Definición del Proceso Organizacional (OPD), Capacitación Organizacional (OT), Administración Integrada de Proyectos (IPM), Administración de Riesgos (RSKM), y Análisis y Resolución de Decisiones (DAR).
- **Nivel 4, Cuantitativamente Administrado:** áreas de **medición estadística**. Performance del Proceso Organizacional (OPP) y Administración Cuantitativa del Proyecto (QPM).
- **Nivel 5, Optimizado:** áreas de **mejora continua**. Administración del Desempeño Organizacional y Análisis Causal y Resolución (CAR).
- **Nivel 1, Inicial:** no tiene áreas de proceso, porque los procesos no están definidos o son improvisados.

CCMI vs ágil

CMMI nació para organizaciones grandes, con procesos definidos, documentados y medidos. Los métodos ágiles (Scrum, XP) nacieron casi como reacción a eso: priorizan las personas, el software funcionando y la respuesta al cambio, por sobre los procesos y la documentación. La pregunta es si una organización ágil puede usar CMMI, o si son incompatibles. Ágil se lleva bien con todo lo que hace el equipo por sí mismo, y choca con todo lo que implique control externo o medición estadística del proceso.

Nivel	Verde (da soporte)	Negro (neutral)	Rojo (desigual)
1	Identificar alcance, realizar el trabajo	—	—
2	Planes, recursos, responsabilidades, capacitación, gestión de configuración	Política organizacional, involucrados, monitorear el plan, revisar con la alta gerencia	**QA objetivo de procesos y productos (PPQA)**
3	—	Mantener un proceso definido	Medir la performance del proceso
4	—	—	**Todo**: objetivos cuantitativos, estabilizar la performance
5	Mejora continua	Causa raíz de defectos	—

Tolerancia de CMMI hacia ágil: como se vio, hay áreas de proceso con soporte, otras neutrales y otras en conflicto. Además, CMMI **admite evaluaciones en ambientes ágiles**. Esto es posible porque CMMI dice **qué** lograr, no **cómo**. Un equipo ágil puede cumplir un objetivo de CMMI con sus propias prácticas.

Tolerancia de ágil hacia CMMI: ¿está la puerta abierta? ¿es posible? Es más difícil, porque muchos defensores de ágil ven a CMMI como la burocracia que justamente quieren evitar.

Lo ideal es un **punto de equilibrio**: ni tanta libertad que se vuelva caos, ni tanta disciplina que se vuelva burocracia.

Aspecto	**CMMI**	**Ágil**
Valor central	Mejores procesos → mejor producto	Respuesta al cliente con mínima sobrecarga
Personas	Disciplinadas, que siguen reglas, con aversión al riesgo	Creativas, cómodas con el cambio, que toman riesgos
Comunicación	Organizacional, formal (macro)	Persona a persona (micro)
Dónde está el conocimiento	En los activos de proceso (documentos, estándares)	En las personas (tradición oral)
Alcance de la mejora	Toda la organización, buscando uniformidad	El proyecto, buscando innovación
Cómo se logra el éxito	Por predictibilidad	Por aprovechar oportunidades
Atajos	Desalentados	Alentados
Quién decide	Comités	Individuos
En qué confía el cliente	En la infraestructura de procesos	En el software funcionando
Enfoque	Cargado al frente: planificar todo antes (*mover a la derecha*)	Conducido por pruebas: probar desde el inicio (*mover a la izquierda*)
Tipo de enfoque	Descriptivo, cuantitativo, universal, estratégico	Prescriptivo, cualitativo, situacional, táctico
Lema	"¿Cómo lo llamaremos?"	"¡Solo hazlo!"
Gestión de riesgos	Proactiva	Reactiva
Foco	Interno, reglas, predictibilidad, estabilidad	Externo, innovación, performance, velocidad

A pesar de las diferencias, comparten bastante:

- **El mismo objetivo:** lograr organizaciones de alto desempeño.
- **Ambos planifican.** Ágil no es "no planificar": planifica en ciclos cortos, en lugar de todo al principio.
- **Ambos tienen reglas,** que son los requerimientos del proceso, y no cumplirlas tiene consecuencias. Un equipo que dice ser Scrum pero no hace retrospectivas no está haciendo Scrum.
- **Ninguno es completo:** ninguno cubre por sí solo todo lo que una organización necesita.
- **No son ideas nuevas:** ambos se basan en la experiencia acumulada de la industria.
- **Ninguno sirve para cualquier proyecto.** Es el mismo principio de "lo que sirve a unos puede no servir a otros".

Conclusión

"¿se puede?" es **sí, con matices**. Son compatibles en gran parte de los niveles bajos y en la idea de mejora continua. Las mayores tensiones están en el control externo del proceso (PPQA) y en la medición cuantitativa de los niveles 3 y 4. Combinarlos implica tomar de cada uno lo que sirve al contexto: la disciplina y la visión organizacional de CMMI, con la flexibilidad y el foco en el cliente de ágil.

TESTING DE SOFTWARE

E**l testing es solo una parte de la calidad**: un tipo de control, sobre el producto. Es lo mismo que mostraba el mapa de "Calidad en el Software", donde el testing aparecía como la única técnica de control de calidad de producto. Tenerlo claro evita el error de pensar que "hacer testing" es lo mismo que "tener calidad".

¿Qué es el testing?

- Es un **proceso destructivo** de tratar de encontrar defectos, cuya **presencia se asume**.
- Se debe ir con una **actitud negativa**, para demostrar que algo es incorrecto.
- **Un testing exitoso es el que encuentra defectos.**
- A nivel mundial, el testing representa entre el **30% y el 50% del costo** de un software confiable.

Testing es el proceso de ejecutar un programa con la intención de encontrar errores. Además, parte de que **todo programa tiene errores**, así que la pregunta no es si los hay, sino dónde están.

De ahí sale la idea de "exitoso". Normalmente, si una prueba no encuentra nada, se dice que "pasó" o que fue exitosa. Myers dice que es al revés y usa una analogía médica. Si alguien se siente mal y le hacen análisis que no encuentran nada, esos análisis no fueron "exitosos": el paciente gastó plata y sigue enfermo. El análisis exitoso es el que **encuentra la enfermedad**, porque permite tratarla. Con el software pasa igual: una prueba que encuentra un defecto agrega valor, porque ese defecto se puede corregir antes de que llegue al usuario.

Testing en el contexto: Asegurar la calidad vs. Controlar la calidad

Una vez definidos los requerimientos de calidad, hay que tener en cuenta que:

- **La calidad no puede "inyectarse" al final.** No se puede construir el sistema de cualquier manera y esperar que el testing lo "arregle" al terminar. Es el primer principio de calidad que vimos.
- **La calidad del producto depende de tareas realizadas durante todo el proceso**: requerimientos, diseño, código y revisiones. El testing solo muestra cómo quedó el resultado de todo eso.
- **Detectar errores en forma temprana ahorra esfuerzo, tiempo y recursos.** Un error de requerimientos encontrado en el testing obliga a rehacer diseño, código y pruebas. Encontrado en una revisión de requerimientos, se corrige en un documento.
- **La calidad no abarca solo el producto, sino también el proceso**, y cómo mejorarlo para evitar **defectos recurrentes**. Es la lógica de "procesos basados en calidad": si el testing encuentra siempre el mismo tipo de defecto, la solución no es seguir encontrándolo, sino corregir el proceso que lo genera.
- **El testing NO puede asegurar ni calidad en el software ni software de calidad.**

El testing es **control**: detecta defectos en un producto ya construido. El **aseguramiento** es preventivo y actúa sobre el proceso. Por eso el testing, por sí solo, no garantiza la calidad. Puede decir que encontró defectos, pero nunca que no quedan más. Es el mismo principio de PPQA: "el aumento de las pruebas no aumenta la calidad".

Error vs. Defecto

- **Error:** el problema se detecta **en la misma etapa en la que se introdujo**. Por ejemplo, el analista escribe mal un requerimiento y lo descubre en la revisión de requerimientos. Es el Homer que se da cuenta solo: el problema no salió de su etapa.
- **Defecto:** el problema **pasa a una etapa posterior** sin ser detectado y se descubre más adelante. Por ejemplo, ese requerimiento mal escrito se diseña, se programa, y recién lo encuentra un tester. Es el Homer señalado por otro, y el problema ya es un *bug* dentro del producto.

Un defecto es mucho más caro que un error. Cuando el problema avanza de etapa, hay que corregirlo en todo lo que se construyó encima: el diseño, el código, las pruebas. Por eso todo el enfoque de calidad busca encontrar los problemas como **errores** (temprano) y no como **defectos** (tarde). Es la razón de ser de las revisiones técnicas.

Conceptos: Defectos, severidad y prioridad

Severidad: qué tan grave es el impacto del defecto en el sistema. Es una mirada **técnica**, y normalmente la asigna el tester.

Nivel	Significado	Ejemplo
1 – Bloqueante	Impide seguir usando o probando el sistema, o una parte entera	La aplicación no arranca; no se puede iniciar sesión
2 – Crítico	Una función principal no anda o da resultados incorrectos, sin alternativa	El sistema calcula mal el total de una factura
3 – Mayor	Una función no anda bien, pero hay una forma alternativa de hacerlo	No funciona el botón de exportar, pero se puede copiar a mano
4 – Menor	Problema chico que no afecta la funcionalidad principal	Un mensaje de error poco claro
5 – Cosmético	Solo afecta la apariencia	Un texto desalineado, un color incorrecto, una falta de ortografía

Prioridad: con qué urgencia hay que corregir el defecto. Es una mirada de **negocio**, y normalmente la decide el líder del proyecto o el cliente.

Nivel	Significado
1 – Urgencia	Se corrige ya, antes que cualquier otra cosa
2 – Alta	Se corrige lo antes posible, en la próxima versión
3 – Media	Se corrige cuando se pueda, en el curso normal del trabajo
4 – Baja	Se corrige si sobra tiempo, o se posterga

Aunque muchas veces van juntas (un defecto bloqueante suele ser urgente), **no siempre coinciden**:

- **Severidad baja, prioridad alta:** el nombre de la empresa del cliente está mal escrito en la pantalla de inicio. Técnicamente es **cosmético**, pero queda muy mal ante el cliente, así que es **urgente**.
- **Severidad alta, prioridad baja:** el sistema se cuelga al generar un reporte que solo se usa una vez al año y para el que falta mucho. Técnicamente es **crítico**, pero puede esperar, porque nadie lo va a usar pronto.

Niveles de Prueba

Cada nivel prueba el software a una **escala distinta**: desde la pieza más chica hasta el sistema completo en manos del usuario. Van en este orden porque cada uno se apoya en el anterior: no tiene sentido probar el sistema entero si sus piezas individuales no funcionan.

Testing Unitario

Qué es: la prueba de cada **componente individual** del software (una clase, un método, una función), de forma **aislada** del resto, apenas se termina de construir.

La filmina lo ilustra con Bernardino Rivadavia, por el juego de palabras: fue el primer presidente **unitario**.

Características:

- Se prueban solo componentes individuales, y cada uno **independientemente** de los demás.
- Se hace **con acceso al código**, usando el entorno de desarrollo: frameworks de pruebas unitarias como JUnit, o herramientas de depuración.
- Normalmente lo hace **el propio desarrollador**.
- Los errores se corrigen **apenas se encuentran, sin registro formal**. Es el desarrollador probando su propio trabajo antes de entregarlo, así que no se abre un reporte de defecto por cada cosa que arregla.
- Se realizan en el workflow de implementación.

Testing de Integración

Qué es: la prueba que verifica que **las partes que funcionan bien por separado también funcionan bien juntas**. Se centra en las **interfaces**, es decir, en cómo se comunican los componentes entre sí.

Por qué hace falta: que dos componentes pasen sus pruebas unitarias no garantiza que funcionen juntos. Uno puede mandar un dato en un formato que el otro no espera, o los dos pueden interpretar distinto una misma regla.

Características:

- La integración debe ser **incremental**: se van conectando las partes de a poco, no todas de golpe. Si se junta todo a la vez y algo falla, es muy difícil saber dónde está el problema.
- Hay dos estrategias principales:
    - **Top-down (de arriba hacia abajo):** se empieza por el módulo principal y se van agregando los de abajo. Los que todavía no están se reemplazan con **stubs**.
    - **Bottom-up (de abajo hacia arriba):** se empieza por los módulos de más bajo nivel y se va subiendo. Los de arriba se simulan con **drivers**.
- Lo ideal es **combinar ambos** esquemas.
- Los **módulos críticos** deben probarse lo antes posible.
- Se realiza en el workflow de implementación.

- Las claves son **conectar de a poco las partes más complejas** y **minimizar los programas auxiliares** (stubs y drivers), porque construirlos también lleva trabajo.

Testing de Sistema

Qué es: la prueba del sistema **completo, funcionando como un todo**, es decir, la prueba de la construcción final.

Características:

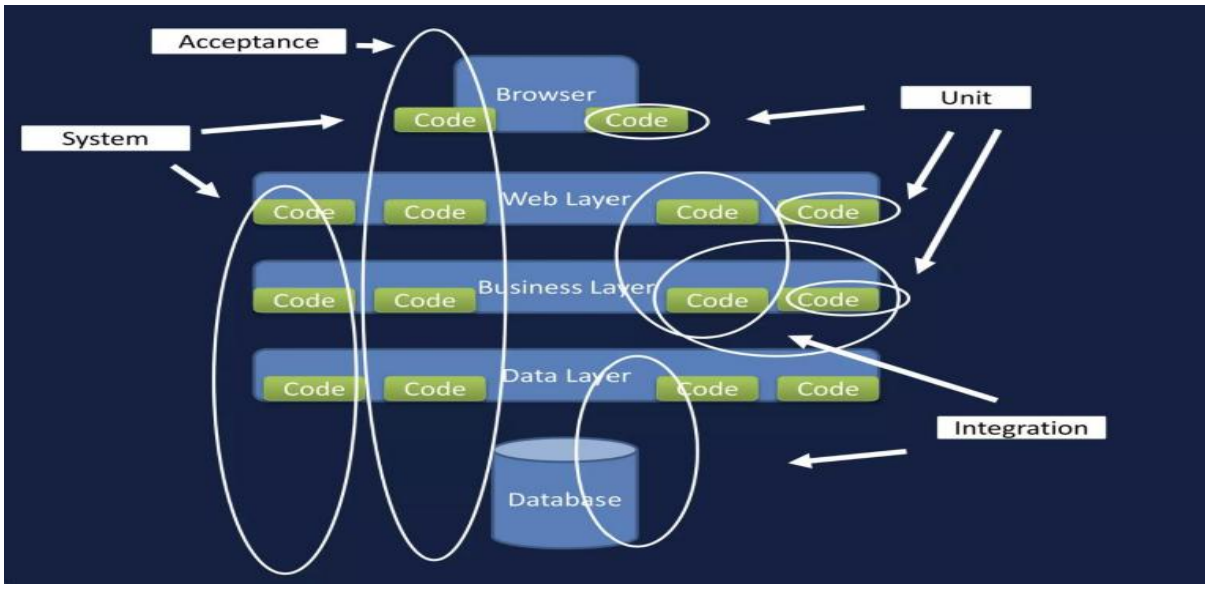
- Busca determinar si el sistema **en su globalidad** opera satisfactoriamente.
- Prueba requerimientos **funcionales** (que haga lo que tiene que hacer) y **no funcionales** (cómo lo hace): recuperación ante fallas, seguridad, stress, performance, etc. Es el nivel donde se prueban la mayoría de los atributos de calidad que vimos en ISO 25010.
- El **ambiente de prueba debe parecerse lo más posible al de producción**, para reducir el riesgo de fallas causadas por diferencias de entorno que no aparecieron en las pruebas. Por ejemplo, algo que anda en la máquina del desarrollador pero falla en el servidor real por otra versión de base de datos.
- Normalmente lo hace un **equipo de testing**, no los desarrolladores.
- Se prueba porciones verticales.

Testing de Aceptación

Qué es: la prueba que hace **el usuario** para determinar si la aplicación **se ajusta a sus necesidades**. El usuario tiene la última palabra sobre si acepta o no el sistema.

Características:

- El objetivo es **establecer confianza** en el sistema, en sus partes o en características específicas.
- **Encontrar defectos NO es el foco principal.** Es la gran diferencia con los otros niveles: acá se busca confirmar que el sistema sirve para lo que el usuario necesita. Si aparecen muchos defectos en esta etapa, es señal de que fallaron los niveles anteriores.
- Tiene dos variantes:
    - **Pruebas alfa:** el usuario prueba en un **ambiente controlado** (laboratorio), normalmente en las instalaciones del desarrollador y con el equipo cerca.
    - **Pruebas beta:** el usuario prueba en su **ambiente real de trabajo**, sin el equipo de desarrollo presente, y reporta los problemas que encuentra.



Los 4 ambientes ideales

Un **ambiente** (o entorno) es el **conjunto de recursos donde se instala y ejecuta el software** en una determinada etapa de su construcción. Incluye:

- **Hardware:** servidores, equipos, red.
- **Software de base:** sistema operativo, base de datos, servidor web, librerías y sus versiones.
- **Configuración:** parámetros, conexiones, permisos.
- **Datos:** de prueba o reales, según el ambiente.
- **La versión del sistema** que está instalada ahí.

¿Por qué se separan en varios ambientes?

Porque cada etapa necesita condiciones distintas, y mezclarlas genera problemas.

1. **Desarrollo:** donde los programadores escriben el código y hacen las pruebas unitarias. Es un ambiente inestable, que cambia todo el tiempo.
2. **Prueba (testing):** donde el equipo de testing ejecuta las pruebas de integración y de sistema, sobre una versión estable que le entregan los desarrolladores. Está separado de desarrollo para que el código no cambie mientras se prueba.
3. **Preproducción:** una **réplica lo más exacta posible de producción**: mismo hardware, sistema operativo, versiones y configuración, y datos parecidos a los reales. Ahí se hacen las pruebas de aceptación y las de performance, y se ensaya la instalación antes de salir. Responde a lo que vimos en testing de sistema: el ambiente de prueba tiene que parecerse al de producción para no tener sorpresas.
4. **Producción:** donde el sistema funciona con usuarios y datos reales.

La cantidad mínima que suele darse es de **3 ambientes: desarrollo, prueba y producción**. Se elimina **preproducción**, y sus funciones las absorbe el ambiente de prueba, que en ese caso conviene configurar lo más parecido posible a producción.

Caso de Prueba

Un caso de prueba es un **conjunto de condiciones o variables bajo las cuales un tester determinará si el software está funcionando correctamente o no**.

Si un tester encuentra un defecto, el desarrollador tiene que poder repetir exactamente los mismos pasos para verlo y corregirlo. Un defecto que no se puede reproducir es muy difícil de arreglar.

"Los bugs se esconden en las esquinas y se congregan en los límites" (Boris Beizer)

La frase indica **dónde buscar**: los defectos suelen aparecer en los casos extremos y en los bordes de los rangos de valores, más que en los casos "normales". Es la base de la técnica de valores límite, que vas a ver en caja negra.

Derivación de casos de prueba

Los casos de prueba se pueden obtener de distintas fuentes:

- Desde los **requerimientos**.
- Desde las **especificaciones de programación**.
- Desde el **código**.
- Desde **documentos del cliente**.
- Desde la **información de relevamiento**.

Condiciones de prueba

Una condición de prueba es la reacción esperada de un sistema frente a un estímulo particular, donde el estímulo son las distintas entradas.

La regla es que cada condición de prueba debe ser probada por al menos un caso de prueba.

Diferencia con el caso de prueba: la condición es qué se quiere verificar, y el caso de prueba es cómo se verifica, con datos concretos. Por ejemplo:

- Condición: "si el usuario ingresa una contraseña incorrecta, el sistema debe rechazar el acceso".
- Caso de prueba: usuario "juan", contraseña "1234" (incorrecta) → resultado esperado: mensaje "Contraseña incorrecta" y no se permite el acceso.

Métodos

¿Para qué usarlos? Porque **el tiempo y el presupuesto son limitados**, y el testing exhaustivo es imposible. Hay que **pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas**.

Es el "mínimo de esfuerzo y tiempo" del principio. Los métodos son las técnicas que permiten elegir **pocos casos de prueba bien elegidos** que tengan la mayor probabilidad de encontrar defectos, en lugar de probar al azar o probarlo todo.

Caja Negra

Se basa en el **comportamiento externo**: se le dan entradas al sistema y se verifica que las salidas sean las esperadas según la especificación, sin mirar el código. Hay dos grupos de métodos.

Basados en especificaciones: derivan los casos de los requerimientos de forma sistemática.

- **Partición de equivalencias:** se dividen los posibles valores de entrada en grupos (clases) que el sistema debería tratar igual, y se prueba **un valor representativo de cada clase**. Si un valor de la clase funciona, se asume que todos los de esa clase funcionan.
- **Análisis de valores límite:** variante de la anterior. En lugar de cualquier valor de la clase, se prueban los **bordes** de cada clase, porque ahí se concentran los defectos (la frase de Beizer).

Caja Blanca

Se basa en el **análisis de la estructura interna** del software o de un componente. Su gran ventaja, es que **se puede garantizar el testing coverage** (cobertura): como se conoce el código, se puede medir qué porcentaje se ejecutó con las pruebas.

Ciclo de Prueba

Un **ciclo de pruebas** es la **ejecución de la totalidad de los casos de prueba** establecidos, **aplicados a una misma versión** del sistema.

Las dos partes de la definición importan:

- **"La totalidad de los casos":** un ciclo no es correr algunas pruebas sueltas, sino todo el conjunto planificado.
- **"Una misma versión":** durante el ciclo, el software no cambia. Si los desarrolladores corrigieran cosas en el medio, los resultados del principio del ciclo ya no valdrían para lo que se está probando al final.

Cómo funciona en la práctica:

1. Los desarrolladores entregan una versión al equipo de testing, por ejemplo la versión 1.0.
2. Se ejecutan todos los casos de prueba sobre esa versión: es el **ciclo 1**.
3. Se registran los defectos encontrados.
4. Los desarrolladores corrigen esos defectos y entregan una nueva versión, la 1.1.
5. Se ejecuta el **ciclo 2** sobre la versión nueva.

Y así hasta que se cumple el criterio de aceptación definido para dejar de probar.

Regresión

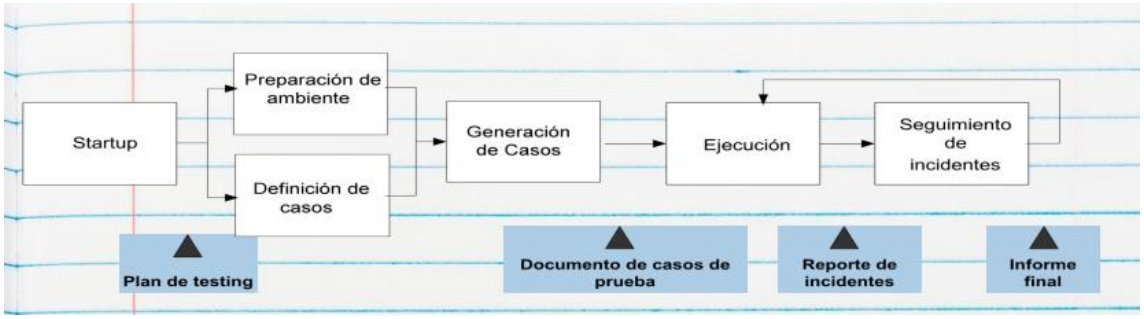
Al terminar un ciclo de pruebas y **reemplazarse la versión** del sistema, debe hacerse una **verificación total de la nueva versión**, para **prevenir la introducción de nuevos defectos al intentar solucionar los detectados**.

El problema que resuelve: cuando un desarrollador corrige un defecto, cambia código. Ese cambio puede romper otras partes del sistema que antes funcionaban, porque el código está conectado. Arreglar una cosa puede descomponer otra, a veces en un lugar que aparentemente no tiene nada que ver.

La **prueba de regresión** consiste en **volver a ejecutar las pruebas que ya habían pasado**, no solo las de los defectos corregidos, para asegurarse de que nada se rompió con los cambios. El nombre viene de que se busca detectar si el sistema **"retrocedió"** (regresó) a un estado peor.

Volver a ejecutar todas las pruebas en cada ciclo lleva mucho tiempo si se hace a mano, y cuanto más crece el sistema, más pruebas hay. Por eso la regresión es la prueba que más se **automatiza**, y es una de las prácticas que vas a ver en testing ágil ("pruebas de regresión automatizadas").

Proceso de Pruebas



1 Planificación de pruebas → Plan de prueba

Planificar es **verificar que se entienden las metas y objetivos del cliente, de los interesados y del proyecto, y los riesgos que las pruebas van a abordar**. Antes de probar, hay que saber **para qué** se prueba y **qué es lo más importante** de probar.

El resultado es el **plan de prueba** (*test plan*), que define:

- **Riesgos y objetivos del testing:** qué se quiere lograr y qué partes son más riesgosas, para probarlas primero y con más esfuerzo.
- **Estrategia de testing:** cómo se va a probar: qué niveles, qué tipos de prueba, qué técnicas.
- **Recursos:** personas, ambientes, herramientas y tiempo.
- **Criterio de aceptación:** cuándo se considera que se probó lo suficiente para terminar.

2 Diseño de casos de prueba → Casos de prueba

- **Revisar la base de pruebas:** los documentos de donde salen las pruebas (requerimientos, casos de uso, especificaciones).
- **Verificar las especificaciones** del software a probar.
- **Evaluar la testeabilidad** de los requerimientos y del sistema. Un requerimiento como "el sistema debe ser rápido" no se puede probar. Hace falta algo medible, como "responder en menos de 2 segundos". Detectarlo acá permite pedir que se aclare.
- **Identificar los datos necesarios** para las pruebas.
- **Diseñar y priorizar los casos de prueba**, aplicando las técnicas de caja negra y caja blanca.
- **Diseñar el entorno de prueba:** cómo tiene que estar configurado el ambiente.

El resultado son los **casos de prueba**, cada uno con sus tres partes: objetivo, datos de entrada y ambiente, y resultado esperado. En la filmina de entregables aparece como el **"Documento de casos de prueba"**.

3 Ejecución

Las actividades son:

- Priorizar los casos de prueba.
- **Crear los datos de prueba.**
- **Automatizar** lo que sea necesario.
- Armar **conjuntos de pruebas** (*test suites*) para ejecutar de forma eficiente.
- **Implementar y verificar el ambiente.**
- **Ejecutar** los casos de prueba.
- **Registrar el resultado** y **la identidad y versión del software** probado.
- **Comparar los resultados reales con los esperados.**

Registrar la **versión** es clave: un defecto encontrado tiene que poder asociarse a la versión exacta en la que apareció. Es el concepto de **ciclo de test**: todas las pruebas sobre una misma versión.

4 Reporte de defectos → Reporte de defectos

Cuando el resultado real no coincide con el esperado, se registra un **defecto**, con su severidad y prioridad, y los pasos para reproducirlo. En la filmina de entregables se llama **"Reporte de incidentes"**.

- **Evaluar los criterios de aceptación:** ¿se cumplió lo definido en el plan?
- **Reportar los resultados** a los interesados.
- **Consolidar** la información de las pruebas realizadas.
- **Verificar los entregables** y que los defectos se hayan corregido.
- **Evaluar cómo resultó el testing** y analizar las **lecciones aprendidas**.

El resultado final es el **informe final** de testing.

Por qué ejecución y reporte son recursivos

Después de ejecutar, si hay defectos:

2. Se **reportan**.
3. Los desarrolladores los **corrigen** y entregan una **nueva versión**.
4. Se vuelve a **ejecutar**, con regresión, sobre esa versión.
5. Si aparecen nuevos defectos, se vuelven a reportar.

Este ciclo se repite hasta que llega el **"momento de dejar de probar"**, que es la segunda decisión del diagrama de la filmina. Ese momento lo define el **criterio de aceptación** del plan de prueba, no el hecho de que "ya no aparezcan defectos".

La planificación y el diseño, en cambio, se hacen **una vez al inicio**. Solo se revisan si algo cambia mucho, por ejemplo si se agregan requerimientos nuevos.

Verificación y Validación

Verificación es comprobar que el producto **cumple con su especificación**. Mira hacia adentro: si lo que se construyó respeta lo que se definió en la etapa anterior. Por ejemplo, si el diseño dice que el sistema calcula el IVA al 21%, verificar es comprobar que el código efectivamente lo calcula al 21%.

Validación es comprobar que el producto **satisface las necesidades reales del usuario**. Mira hacia afuera: si lo que se construyó sirve para lo que el cliente realmente quiere. Siguiendo el ejemplo, validar es comprobar que el cliente efectivamente necesitaba calcular el IVA al 21%, y no, por ejemplo, que vende productos con alícuotas distintas.

¿Cuánto testing es suficiente?

El testing exhaustivo es imposible. Entonces, ¿cómo se decide cuánto probar?

Si no se puede probar todo, hay que **elegir**. La decisión depende de dos cosas:

- **La evaluación del nivel de riesgo:** qué tan grave sería que una parte del sistema falle.
- **Los costos asociados al proyecto:** cuánto tiempo y dinero hay disponible.

Los **riesgos** se usan para decidir tres cosas:

- **Qué testear primero:** lo más riesgoso.
- **A qué dedicarle más esfuerzo:** las funciones críticas, por ejemplo el cobro en un sistema de ventas, se prueban con mucha más profundidad que una pantalla de "acerca de".
- **Qué no testear, por ahora:** lo de bajo riesgo puede quedar para después, o no probarse.

Esto se llama **testing basado en riesgos**: el esfuerzo de prueba es proporcional al riesgo.

Para decidir **cuándo terminar** una fase de testing se usa el **criterio de aceptación** (también llamado criterio de salida), que se define en el plan de prueba. Puede expresarse en términos de:

- **Costos:** se prueba hasta agotar el presupuesto asignado.
- **Porcentaje de tests corridos sin fallas:** por ejemplo, el 95% de los casos pasan.
- **Fallas predichas que aún permanecen:** se estima cuántos defectos quedan, en base a datos históricos, y se termina cuando esa cifra baja de un límite.
- **Ausencia de defectos de cierta severidad:** por ejemplo, no quedan defectos bloqueantes ni críticos abiertos.

Lo importante es que **se define de antemano**. Si no, el testing termina cuando se acaba el tiempo, y no cuando se alcanzó el nivel de calidad buscado.

Principios del Testing

1. **El testing muestra la presencia de defectos.** Puede demostrar que **hay** defectos, pero nunca que **no los hay**. Si las pruebas no encuentran nada, eso no prueba que el software esté libre de errores: solo que esas pruebas no los encontraron.
2. **El testing exhaustivo es imposible.** Es lo que mostró el ejemplo de las 240.000 pruebas. Por eso se prioriza por riesgo.
3. **Testing temprano.** Cuanto antes empiece el testing en el ciclo de vida, más barato es corregir lo que se encuentra. Es el modelo en V.
4. **Agrupamiento de defectos.** Los defectos no se distribuyen de forma pareja: **la mayoría se concentra en pocos módulos**. Si un módulo ya tuvo muchos defectos, probablemente tenga más, así que conviene probarlo más. *Complemento (Myers):* lo plantea como "la probabilidad de que existan más errores en una sección es proporcional a la cantidad de errores ya encontrados en ella".
5. **Paradoja del pesticida.** Si se ejecutan siempre las mismas pruebas, con el tiempo **dejan de encontrar defectos nuevos**, igual que los insectos que se vuelven resistentes a un pesticida. Las pruebas tienen que revisarse y renovarse para seguir siendo efectivas.
6. **El testing depende del contexto.** No se prueba igual un videojuego que el software de un marcapasos o un sistema bancario. El tipo de sistema define cuánto, cómo y qué probar.
7. **Falacia de la ausencia de errores.** Encontrar y corregir todos los defectos **no sirve si el sistema no satisface las necesidades del usuario**. Un sistema sin defectos que no le sirve a nadie es un fracaso. Es la diferencia entre verificación y validación.

Tipos de pruebas

Testing funcional

Prueba **qué hace** el sistema. Se basa en las **funciones y características** del sistema, descritas en los documentos o entendidas por los testers, y en su **interoperabilidad** con sistemas específicos. Puede basarse en:

- **Requerimientos:** se prueba cada requerimiento funcional. Ej.: "el sistema debe permitir dar de alta un cliente".
- **Procesos de negocio:** se prueban los flujos completos como los haría el usuario. Ej.: dar de alta un cliente, cargarle un pedido, facturarlo y cobrarlo.

Testing no funcional

Prueba **cómo funciona** el sistema. Un sistema que hace todo bien, pero tarda dos minutos en cada pantalla, se cae con 50 usuarios o nadie entiende cómo usar, también fracasa.

Las características que se prueban:

Prueba	Qué verifica	Ejemplo
Performance	Tiempos de respuesta y concurrencia, en condiciones normales	La búsqueda responde en menos de 2 segundos
Carga	El comportamiento con la **cantidad esperada** de usuarios o transacciones	El sistema soporta 500 usuarios simultáneos, como se espera en un día normal
Stress	El comportamiento **más allá del límite** esperado, hasta que falla, y cómo se recupera	Qué pasa con 5.000 usuarios simultáneos: si se degrada de a poco o se cae de golpe, y si se recupera
Volumen	El manejo de **grandes cantidades de datos**	El reporte anual funciona con una base de 10 millones de registros
Usabilidad	Qué tan fácil es de aprender y de usar	Un usuario nuevo puede completar una compra sin ayuda
Configuración	Que funcione en las distintas combinaciones de entorno: drivers de video, conexiones de red, software de terceros, sistemas operativos, bases de datos, middleware	Que funcione en Windows y Linux, y con distintas versiones de base de datos
Instalación	Que se pueda instalar, actualizar y desinstalar correctamente	La actualización a la nueva versión no pierde datos
Mantenimiento	Qué tan fácil es modificar el sistema	Cuánto lleva hacer un cambio chico
Fiabilidad	Que funcione sin fallar durante un tiempo determinado	El sistema funciona 30 días seguidos sin caerse
Portabilidad	Que se pueda llevar a otro entorno	La aplicación se puede migrar a otro servidor

Testing Ágil

Manifesto del Testing

Pruebas durante el desarrollo sobre pruebas al final

Qué significa: en el enfoque tradicional, el testing es una **fase** que empieza cuando termina el desarrollo, y el trabajo se acumula en la columna de test esperando al tester. En ágil, el testing es una **actividad** que ocurre todo el tiempo, en paralelo con el desarrollo y dentro de cada iteración. Una funcionalidad no se "pasa a test": se prueba mientras se construye. Es el principio de "testing temprano" llevado al extremo.

Prevenir defectos sobre encontrar defectos

Qué significa: en lugar de esperar a que el software esté hecho para buscarle defectos, el tester participa desde que se define la historia, haciendo preguntas que aclaren los requerimientos y los casos borde antes de programar. Muchos defectos vienen de requerimientos ambiguos. Si se aclaran antes, el defecto nunca llega a existir.

Entender lo que se está probando sobre verificar la funcionalidad

Qué significa: un *checker* solo sigue una lista y marca si cada paso pasó o falló, sin entender para qué sirve la funcionalidad. Un *tester* entiende qué necesita el usuario y por qué. Eso le permite encontrar problemas que ninguna lista contempla, y darse cuenta si algo "funciona" pero no sirve. Se relaciona con la diferencia entre verificación y validación: no alcanza con verificar que se cumple el paso a paso, hay que entender si el sistema sirve.

Construir un mejor sistema sobre romper el sistema

Qué significa: el objetivo del tester no es demostrar que puede romper el software, sino colaborar con el equipo para que el producto sea lo mejor posible. El tester no es el adversario del desarrollador, sino parte del mismo equipo con el mismo objetivo. Se conecta con el mito de que "el tester es el enemigo del programador".

Responsabilidad de todo el equipo por la calidad sobre el tester responsable de la calidad

Qué significa: en el enfoque tradicional, el tester es la "aduana" por la que tiene que pasar el producto antes de salir, y si algo falla en producción, la culpa es del tester. En ágil, la calidad es responsabilidad de todo el equipo: desarrolladores, tester y Product Owner deciden juntos cuándo el producto está listo. Es el principio de calidad "es un esfuerzo de todos" que vimos en PPQA.

Los cinco valores describen el mismo cambio: el testing deja de ser una etapa final, a cargo de una persona, que busca romper el sistema y encontrar defectos, y pasa a ser una actividad continua, de todo el equipo, que busca entender el producto y prevenir defectos.

Ciclo de vida del testing vs. Ágil

El ciclo tradicional de testing es un proceso **propio y separado**, con muchas etapas formales que se recorren una vez. En ágil, el testing no tiene un ciclo aparte: es **una parte más del ciclo corto de desarrollo**, y se repite en cada vuelta junto con todo lo demás.

la cascada vs. el ciclo iterativo

Las etapas son las mismas. Lo que cambia es **la duración y la repetición**.

- En cascada, todo el sistema pasa por cada etapa una vez, y el ciclo completo dura **meses o años**. El testing llega recién al final, y el usuario ve el producto cuando ya está todo hecho.
- En ágil, se construye un pedazo chico del sistema, que recorre todas las etapas (incluido el testing) en **días o semanas**. Después se repite con el pedazo siguiente.

las actividades dentro de una iteración

la diferencia más importante del tema, y la misma idea del manifiesto: “testing is an activity, not a phase”.

- En cascada, el testing es una **fase con principio y fin**, que empieza cuando termina la codificación.
- En ágil, codificar, corregir y probar ocurren **al mismo tiempo**, durante toda la iteración. Al final de cada iteración hay un producto probado que se puede entregar.

	Tradicional (cascada)	**Ágil**
Ubicación del testing	Una fase al final, después de codificar	Una actividad continua, durante toda la iteración
Duración del ciclo	Meses o años	Días o semanas
Cuántas veces se prueba	Una vez, al final, todo junto	En cada iteración, de a pedazos
Cuándo se detectan los defectos	Tarde, cuando corregirlos es caro	Enseguida, cuando corregirlos es barato
Relación con el desarrollo	Secuencial: primero se codifica, después se prueba	En paralelo: se codifica, se corrige y se prueba a la vez

En cascada el testing es una fase; en ágil es una actividad.

9 principios del testing ágil

1. El testing se mueve hacia adelante en el proyecto

En cascada, el testing estaba al **final** del proyecto. En ágil se mueve hacia el **comienzo**: empieza cuando se define la historia de usuario, antes de programar. Se pregunta cómo se va a probar algo **antes** de construirlo.

2. El testing no es una fase

Es la idea central del tema anterior. El testing no es una etapa con principio y fin que espera a que termine el desarrollo. Es una **actividad continua** que ocurre en paralelo con la codificación, durante toda la iteración.

3. Todos hacen testing

En el enfoque tradicional, probar era tarea exclusiva del tester. En ágil, **todo el equipo prueba**.

4. Reducir la latencia del feedback

Latencia es el tiempo que pasa entre que se comete un error y que alguien se entera. En cascada, esa latencia era de meses: un error de requerimientos se descubría en la fase de testing. En ágil se busca que sea de **minutos u horas**.

5. Las pruebas representan expectativas

Una prueba no es solo una verificación técnica: es la forma concreta de expresar **qué espera el cliente** del sistema.

6. Mantener el código limpio, corregir los defectos rápido

En el enfoque tradicional, los defectos se acumulaban en una lista que crecía y se corregía al final, o nunca. En ágil, cuando se encuentra un defecto, **se corrige de inmediato**, dentro de la misma iteración.

7. Reducir la sobrecarga de documentación de las pruebas

En el enfoque tradicional, el testing generaba mucha documentación: planes extensos, casos de prueba detallados paso a paso, reportes formales. En ágil se reduce al mínimo necesario.

8. Las pruebas son parte del "done"

En Scrum, una historia está terminada ("done") cuando cumple la **definición de hecho** acordada por el equipo. Este principio dice que **una historia no está terminada si no está probada**.

9. De probar al final a conducido por pruebas

Es el cambio más profundo. En lugar de escribir el código y **después** probarlo, se escriben **primero las pruebas** y después el código necesario para que pasen. Las pruebas pasan a **guiar el desarrollo**.

6 Prácticas Concretas

1. Pruebas de unidad e integración automatizadas

Los desarrolladores escriben **pruebas automatizadas** para cada componente (unitarias) y para la comunicación entre componentes (integración), y las ejecutan una herramienta, por ejemplo JUnit.

2. TDD (*Test Driven Development*, desarrollo guiado por pruebas)

Se escribe **la prueba antes que el código**. La imagen de la filmina muestra el ciclo de tres pasos, conocido como **"rojo, verde, refactor"**:

1. **Rojo:** se escribe una prueba para una funcionalidad que todavía no existe. Se ejecuta y **falla**, en rojo, porque no hay código que la haga pasar.
2. **Verde:** se escribe el **código mínimo** necesario para que la prueba pase, en verde. No más que eso.
3. **Refactor:** se **mejora el código** (se ordena, se eliminan duplicaciones, se hace más claro) **sin cambiar su comportamiento**. Las pruebas garantizan que la mejora no rompió nada.

3. Pruebas de regresión a nivel de sistema automatizadas

Es el concepto de **regresión** que vimos en Testing de Software, aplicado a ágil. Después de cada cambio, se vuelven a ejecutar **todas** las pruebas del sistema completo, para verificar que lo que ya funcionaba siga funcionando.

4. ATDD (*Acceptance Test Driven Development*, desarrollo guiado por pruebas de aceptación)

Es la misma idea que TDD, pero a nivel de **pruebas de aceptación**, es decir, desde el punto de vista del **usuario**. Antes de programar una historia, el equipo **junto con el cliente o el Product Owner** define las pruebas de aceptación que la historia tiene que pasar para considerarse terminada.

5. Pruebas exploratorias

El tester **explora el sistema sin un guion fijo**: aprende cómo funciona mientras lo prueba, y decide la próxima prueba según lo que va encontrando. Diseña y ejecuta las pruebas **al mismo tiempo**. La imagen de la filmina es un mapa del tesoro con un bicho marcado: el tester "explora el territorio" buscando defectos.

6. Control de versión de las pruebas con el código

Las pruebas se guardan **en el mismo repositorio que el código fuente**, y se versionan junto con él. La imagen muestra un repositorio con tres cosas adentro: código fuente, pruebas unitarias y pruebas de sistema.

El rol del Tester en los equipos ágiles

En ágil, el tester **no trabaja aislado** al final de la cadena esperando que le pasen algo para probar. Es un **puente** entre las distintas partes del equipo, y colabora con todas durante todo el sprint.

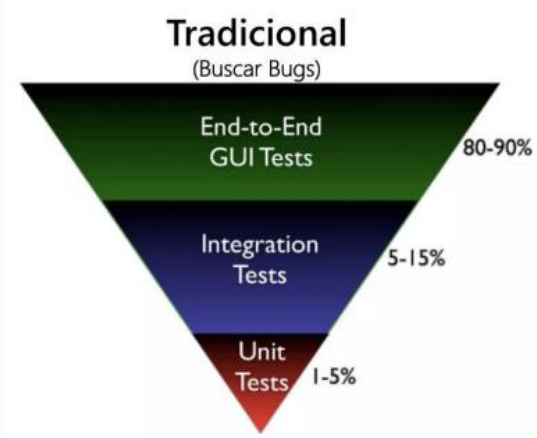
Con el Product Owner: Acá el tester trabaja en la prevención: los problemas que encuentra en una historia antes de programarla nunca llegan a ser defectos. Es el valor del manifiesto "prevenir defectos sobre encontrar defectos".

Con los desarrolladores: Es el valor del manifiesto "construir un mejor sistema sobre romper el sistema": el tester y el desarrollador trabajan juntos, no como adversarios.

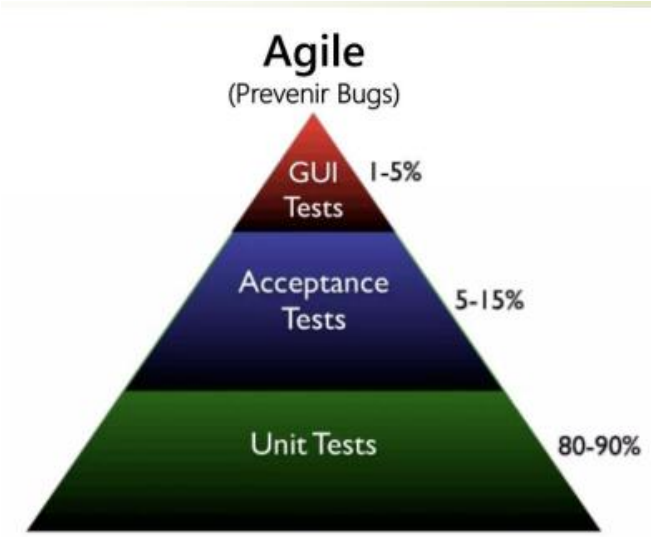
Con el System Team:

- Prepara la documentación de las historias a integrar y probar.
- Difunde el plan, los casos y los datos de prueba para la integración.
- Ayuda a conocer la completitud del feature: si todas las partes de una funcionalidad grande están listas.
- Recibe bugs y los comunica al PO y a los desarrolladores: hace de nexo cuando aparecen problemas en la integración.

Pirámide del Testing Tradicional vs. Agile



Qué significa: en el enfoque tradicional, la gran mayoría de las pruebas se hacen al final, sobre el sistema completo y a través de la interfaz gráfica, muchas veces de forma manual. Casi no hay pruebas unitarias. Por eso el subtítulo dice "Buscar bugs": el testing se centra en encontrar defectos en el producto ya terminado.



Qué significa: la gran mayoría de las pruebas son unitarias, escritas por los desarrolladores (muchas veces con TDD), y automatizadas. En el medio hay pruebas de aceptación, generalmente sobre la capa de servicios o API y no sobre la interfaz. Arriba quedan muy pocas pruebas por interfaz gráfica, solo para los flujos más importantes. Por eso el subtítulo dice “Prevenir bugs”: los defectos se detectan apenas se escribe el código, antes de que lleguen al sistema completo.

Las dos pirámides tienen los mismos porcentajes, pero invertidos: lo que en una es el 80-90%, en la otra es el 1-5%. La regla general es: cuanto más abajo en la pirámide, más rápida, barata y estable es la prueba. Por eso ágil pone ahí la mayoría.

Ojo, que la pirámide ágil no elimina las pruebas de interfaz ni las de aceptación: las reduce a las necesarias. Probar el sistema completo como lo usa el usuario sigue siendo importante, pero no puede ser la base de toda la estrategia.

¿Qué es automatizar el testing?

Automatizar es usar **herramientas y scripts para que las pruebas se ejecuten solas**, sin que una persona tenga que hacerlas a mano paso a paso. La prueba se escribe una vez, en código o con una herramienta, y después se puede ejecutar cuantas veces se quiera, comparando automáticamente el resultado real con el esperado.

Por qué es necesaria en ágil

Los equipos ágiles entregan software listo para producción **al final de cada iteración corta**. Eso exige ejecutar **toda la regresión al menos una vez por día**, y eso **no se puede hacer manualmente**. A medida que el sistema crece, probar todo a mano lleva cada vez más tiempo, hasta que el testing no puede seguirle el ritmo al desarrollo.

Por eso la automatización es la base de varias prácticas que ya vimos: las **pruebas de unidad e integración automatizadas**, la **regresión de sistema automatizada**, el **TDD** y el **ATDD**.

Beneficios de la automatización

1. **Rápida ejecución**
   Una prueba automatizada se ejecuta en segundos o milisegundos. Lo que a una persona le llevaría días, una herramienta lo hace en minutos. Esto permite ejecutar todas las pruebas cada vez que alguien cambia el código, y así **reducir la latencia del feedback**: el desarrollador se entera enseguida si rompió algo.
2. **Reusabilidad**
   La prueba se escribe **una vez** y se reutiliza **en cada versión** del sistema. Es lo que hace viable la **regresión**: en lugar de rehacer manualmente todas las pruebas en cada ciclo, se vuelven a ejecutar las mismas pruebas automatizadas.
3. **Mayor cobertura**
   Como ejecutarlas es rápido y barato, se pueden tener **muchas más pruebas** y cubrir más funcionalidades, más combinaciones de datos y más casos borde de los que se podrían probar a mano.
4. **Mayor precisión**
   Una herramienta ejecuta la prueba **exactamente igual cada vez**.
5. **Mayor alcance**
   Permite hacer pruebas que **a mano serían imposibles o muy difíciles**.
6. **Reducción de costos**
   Automatizar tiene un **costo inicial alto**, porque hay que escribir las pruebas y aprender las herramientas. Pero después, cada ejecución es casi gratis.

LEAN Y KANBAN

Filosofía Lean

¿qué es Lean?

Lean nace en la **industria automotriz japonesa**, en el sistema de producción de **Toyota**, a mediados del siglo XX. La idea central es **producir solo lo que agrega valor al cliente y eliminar todo lo demás**.

Principios

1. Eliminar desperdicios

Es el principio central de Lean. **Desperdicio es todo lo que no agrega valor al cliente**: lo que consume tiempo, esfuerzo o dinero, pero que el cliente no percibe ni pagaría. En software, por ejemplo: funcionalidades que nadie usa, documentación que nadie lee, esperas entre etapas o defectos que hay que corregir.

El objetivo es **identificar** esos desperdicios y **eliminarlos**, para que todo el esfuerzo vaya a lo que sí genera valor. Es el tema de las filminas siguientes, con los tipos de desperdicio.

2. Amplificar el aprendizaje

Desarrollar software no es como fabricar autos en serie: cada producto es nuevo, y no se sabe todo de antemano. Por eso hay que organizar el trabajo para **aprender lo más rápido posible**: iteraciones cortas, feedback frecuente del cliente, pruebas tempranas,

prototipos. Cuanto antes se descubre que algo no funciona o no es lo que el cliente quería, más barato es cambiarlo.

Se relaciona con el principio de testing ágil “reducir la latencia del feedback”.

3. Embeber la integridad conceptual

Embeber significa “incorporar adentro”: la calidad tiene que estar **integrada en el producto desde el principio**, no agregarse al final. **Integridad conceptual** significa que el sistema es **coherente**: sus partes funcionan bien juntas, el diseño es consistente y el usuario lo percibe como un todo armónico.

Es la misma idea que vimos en PPQA: “la calidad no se inyecta, debe estar embebida”. En la práctica se logra con revisiones, refactorización, pruebas automatizadas desde el inicio y TDD.

4. Diferir compromisos

Las decisiones importantes, sobre todo las difíciles de revertir, se toman **lo más tarde posible**, en el **último momento responsable**. Es decir, cuando ya hay suficiente información para decidir bien, pero antes de que no decidir genere problemas.

Por qué: si se decide todo al principio, se decide con poca información, y es probable equivocarse. Postergar la decisión permite aprender más mientras tanto (se relaciona con el principio 2). Por ejemplo, no elegir el diseño definitivo de la base de datos el primer día, sino cuando ya se entienden bien los datos que se van a manejar.

Ojo: diferir **no significa no planificar ni dejar todo para último momento**. Significa no cerrar opciones antes de tiempo.

5. Dar poder al equipo

Las personas que hacen el trabajo son las que **mejor saben cómo hacerlo**. Por eso se les da **autonomía** para tomar decisiones sobre su trabajo, en lugar de que todo lo decida un jefe. El rol del líder pasa a ser apoyar al equipo, darle recursos y sacarle obstáculos, no controlarlo.

Se relaciona con los equipos autoorganizados de ágil y con el principio de calidad “las personas son la clave”.

6. Ver el todo

Hay que **optimizar el sistema completo**, no cada parte por separado. Mejorar una etapa aislada puede empeorar el resultado total. Por ejemplo, si los desarrolladores producen código más rápido pero testing no da abasto, solo se acumula trabajo esperando a ser probado: el sistema entero no entrega más rápido, solo se mueve el cuello de botella.

Es la misma idea del óptimo local vs. global de Barbacci, y es clave en Kanban, que mira el **flujo de punta a punta**.

7. Entregar lo antes posible

Entregar valor al cliente **rápido y seguido**, en entregas chicas y frecuentes. Esto tiene varias ventajas: el cliente obtiene valor antes, se recibe feedback real antes (principio 2),

y hay menos trabajo a medio terminar acumulado, que en Lean es desperdicio (principio 1).

Entregar rápido **no significa apurarse ni bajar la calidad**. Significa reducir el tiempo entre que se pide algo y que se entrega, eliminando esperas y trabajo innecesario.

Los siete principios se refuerzan entre sí. **Eliminar desperdicios** es el objetivo central. **Entregar rápido** y **amplificar el aprendizaje** generan feedback temprano. **Diferir compromisos** permite usar ese aprendizaje para decidir mejor. **Dar poder al equipo** y **ver el todo** hacen que las mejoras beneficien al sistema completo. Y **embeber la integridad** asegura que la velocidad no sacrifique la calidad.

7 + 1 Desperdicios

No todo desperdicio se puede eliminar. Se distingue dos tipos:

- **Tipo 1, desperdicio necesario → minimizar.** No agrega valor directo al cliente, pero **hoy no se puede eliminar**, porque algo lo exige: una ley, un contrato, una limitación técnica. Por ejemplo, cierta documentación exigida por una norma, o la aprobación formal de un cliente. La estrategia es **reducirlo al mínimo**.
- **Tipo 2, puro desperdicio → evitar.** No agrega valor y **nada lo justifica**. Por ejemplo, esperar días a que alguien apruebe algo que no necesita aprobación, o desarrollar una funcionalidad que nadie pidió. La estrategia es **eliminarlo**.

Los 7 desperdicips son:

1. Inventario → Trabajo parcialmente terminado
En una fábrica, el inventario son piezas almacenadas que todavía no se vendieron: ocupan lugar, inmovilizan dinero y pueden volverse obsoletas. En software es el **trabajo empezado y no terminado**: código escrito pero no probado, funcionalidades a medio hacer, requerimientos analizados que nadie programó. No le da valor a nadie hasta que se termina, y mientras tanto puede quedar desactualizado. Es la razón por la que Kanban **limita el trabajo en curso (WIP)**.

2. Sobreproducción → Características extra
En una fábrica, es producir más de lo que se va a vender. En software, es **desarrollar funcionalidades que el cliente no pidió o no usa**. Es considerado uno de los peores desperdicios, porque además de costar desarrollarlas, después hay que probarlas, mantenerlas y documentarlas, y hacen el sistema más complejo.

3. Proceso extra → Proceso extra
Es igual en los dos casos: **pasos o tareas que no agregan valor**. En software: documentación que nadie lee, aprobaciones innecesarias, reportes duplicados, reuniones sin propósito. Ojo con el Tipo 1: parte de esto puede ser necesario, por ejemplo por exigencia de un cliente. En ese caso se minimiza.

4. Transporte → Cambio de tareas y transferencia de conocimiento
En una fábrica, es mover materiales de un lugar a otro sin transformarlos. En software, lo que se "transporta" es el **conocimiento**: cada vez que el trabajo pasa de una persona a

otra (del analista al desarrollador, del desarrollador al tester), se pierde información en el camino y hay que volver a explicar. Cuantos más traspasos, más pérdida.

5. Esperas → Esperas / Demoras
Igual en los dos casos: **tiempo en que el trabajo está detenido** esperando algo. En software: esperar que el cliente responda una consulta, que se apruebe un cambio, que se libere un ambiente de pruebas, que otro equipo termine su parte. Mientras espera, el trabajo no avanza, pero el tiempo del cliente sí corre.

6. Movimiento → Cambio de contexto (seteo)
En una fábrica, son los movimientos innecesarios de los operarios, como caminar a buscar una herramienta. En software, es **saltar de una tarea a otra**: cada vez que una persona deja lo que estaba haciendo para atender otra cosa, pierde tiempo en "volver a entrar" en la tarea, es decir, en recordar dónde estaba y en qué pensaba. Se llama "seteo" porque es como reconfigurar una máquina cada vez que se cambia de producto. Trabajar en muchas cosas a la vez es más lento que terminarlas de a una.

7. Defectos → Defectos
Igual en los dos casos: los **errores que hay que corregir**. En software, cada defecto implica encontrarlo, entenderlo, corregirlo y volver a probar. Y cuanto más tarde se encuentra, más caro es, como vimos en calidad. Es un desperdicio porque ese esfuerzo de corrección no habría sido necesario si se hacía bien la primera vez.

+1. Talento no utilizado
Se agregó después a los siete originales. Es **no aprovechar las capacidades, ideas y conocimientos de las personas**: tener gente capaz haciendo solo tareas mecánicas, o no escuchar sus propuestas de mejora. Se relaciona con el principio Lean de **dar poder al equipo**.

KANBAN

Kanban es un **método para definir, gestionar y mejorar servicios** que entregan **trabajo del conocimiento**, tales como servicios profesionales, trabajos o actividades en las que interviene la creatividad y el diseño, tanto de productos de software como físicos.

Vale la pena desarmarla por partes:

- **"Un método":** Kanban no es una metodología completa que dice todo lo que hay que hacer, con roles, ceremonias y artefactos, como Scrum. Es un **método** que se aplica **sobre la forma de trabajo que ya existe**, para mejorarla.
- **"Definir, gestionar y mejorar":** son tres acciones. Primero se **define** cómo funciona el trabajo, haciéndolo visible. Después se **gestiona** el flujo día a día. Y se **mejora** de forma continua.
- **"Servicios":** Kanban ve el trabajo como un **servicio** que se le presta a un cliente, interno o externo. El foco está en **entregarle valor** a ese cliente.
- **"Trabajo del conocimiento":** trabajo **intelectual**, donde lo que se produce son ideas, diseños, código o análisis, y no objetos físicos en una línea de montaje.

Este tipo de trabajo es **invisible**: no se ve una pila de piezas acumulándose como en una fábrica.

Valores de Kanban

1. **Transparencia:** compartir la información abiertamente mejora el flujo de valor. Incluye usar un lenguaje claro y directo. Es la base del tablero kanban: que todos vean el estado real del trabajo.
2. **Equilibrio:** hay que balancear distintos aspectos, puntos de vista y capacidades para ser efectivos. El ejemplo clave es **demanda vs. capacidad**: si entra más trabajo del que el equipo puede hacer durante mucho tiempo, el sistema colapsa. Por eso se limita el WIP.
3. **Colaboración:** trabajar juntos. Kanban se creó justamente para mejorar la forma en que las personas trabajan en conjunto.
4. **Foco en el cliente:** todo sistema kanban apunta a un punto donde se entrega valor, que es cuando el cliente recibe lo que pidió. El cliente puede ser interno o externo a la organización.
5. **Flujo:** el trabajo es un flujo de valor, continuo o puntual. **Ver el flujo** es el punto de partida esencial para usar Kanban.
6. **Liderazgo:** la capacidad de inspirar a otros a actuar, con el ejemplo, la palabra y la reflexión. En Kanban hace falta liderazgo **en todos los niveles**, no solo en los jefes.
7. **Entendimiento:** conocerse a uno mismo, como persona y como organización, para poder avanzar. Como Kanban es un método de mejora, **conocer el punto de partida** es la base de todo.
8. **Acuerdo:** el compromiso de avanzar juntos hacia los objetivos, respetando las diferencias de opinión. El libro aclara que no es “gestión por consenso” (que todos estén de acuerdo en todo), sino un compromiso compartido de mejorar.
9. **Respeto:** valorar, entender y tener consideración por las personas. Está al final de la lista porque es **la base sobre la que se apoyan todos los demás**.

Principios de Kanban

Principios de gestión del cambio

1. Comenzar con lo que haces ahora

- **Entender los procesos actuales**, tal como se están haciendo hoy.
- **Respetar los roles, responsabilidades y puestos de trabajo actuales.**

2. Acordar la búsqueda de la mejora a través del cambio evolutivo
La mejora se hace con **cambios pequeños y graduales**, no con una revolución. Cada cambio se prueba, se evalúa y, si funciona, se mantiene. Así el riesgo es bajo y la gente se adapta de a poco. La palabra "acordar" es importante: todos se comprometen a mejorar continuamente.

3. Fomentar actos de liderazgo en todos los niveles
Cualquier persona, desde un desarrollador hasta un gerente, puede proponer y liderar mejoras. No hay que esperar a que la mejora venga de arriba. Se relaciona con el valor **liderazgo**.

Principios de entrega de servicios

4. Comprender y enfocarse en cumplir las necesidades y expectativas del cliente
El objetivo del servicio es **satisfacer al cliente**. Hay que entender qué necesita y qué espera, por ejemplo en tiempos de entrega o en calidad, y organizar el trabajo en función de eso.

5. Gestionar el trabajo; dejar que los trabajadores se autoorganicen en torno a él
Lo que se gestiona es **el trabajo y su flujo**, no a las personas. No se trata de controlar qué hace cada uno ni de mantener a todos ocupados, sino de que el trabajo fluya. Las personas se **autoorganizan** para decidir cómo hacerlo. Se relaciona con el principio Lean "dar poder al equipo".

6. Revisar periódicamente la red de servicios y sus políticas para mejorar los resultados
Las reglas de trabajo (las **políticas**, como los límites de WIP o cuándo una tarea se considera terminada) **no son fijas**. Se revisan regularmente, y se ajustan si se descubre que otra forma da mejores resultados al cliente.

Prácticas Generales de KANBAN

1. Visualizar

Hacer **visible** el trabajo y el proceso, normalmente en un **tablero kanban**: columnas que representan las etapas del proceso y tarjetas que representan cada ítem de trabajo, moviéndose de izquierda a derecha.

2. Limitar el WIP (trabajo en progreso)

Asignar **límites explícitos** a cuántos ítems puede haber **en progreso** en cada etapa del flujo. Por ejemplo, la columna "Desarrollo" tiene límite 4: si ya hay 4 tarjetas, no puede entrar una nueva hasta que salga una.

Limitar el WIP transforma el sistema de **push** (empujar trabajo) a **pull** (tirar trabajo): no se empieza algo nuevo hasta terminar lo anterior. Es la idea del supermercado de

Toyota. Además, ataca directamente el desperdicio de **trabajo parcialmente terminado**.

3. Gestionar el flujo

El objetivo es que el trabajo **termine de la forma más fluida y predecible posible, manteniendo un ritmo sostenido**.

4. Hacer explícitas las políticas

Las **políticas** son las **reglas de cómo se trabaja**: cuándo una tarea se considera terminada en cada etapa, cuáles son los límites de WIP, cómo se prioriza, quién puede tomar un ítem, etc. Hacerlas **explícitas** significa escribirlas y dejarlas a la vista, en lugar de que estén en la cabeza de algunos.

5. Implementar ciclos de retroalimentación

Las **reuniones y revisiones periódicas** para revisar el trabajo y mejorar. La más conocida es la **reunión diaria de Kanban**, frente al tablero, enfocada en terminar los ítems y destrabar los bloqueados.

6. Mejorar colaborativamente, evolucionar experimentalmente

Es la práctica que pone en acción los principios de gestión del cambio.

Cómo funciona:

1. Se plantea una **hipótesis** de mejora. Por ejemplo: "si bajamos el límite de WIP de desarrollo de 5 a 3, los ítems van a terminar más rápido".
2. Se diseña un **experimento** en un entorno **donde fallar es seguro**.
3. Si la hipótesis es correcta y el experimento da buenos resultados, **se mantiene el cambio**.
4. Si el resultado no es positivo, **se puede volver fácilmente al estado anterior**.

Las prácticas siguen una lógica: primero se **ve** el trabajo (visualizar), después se **controla** cuánto entra (limitar WIP) y cómo avanza (gestionar el flujo), con **reglas claras** (políticas), se **revisa** periódicamente (retroalimentación) y se **mejora** de a poco (mejora colaborativa). El ciclo se repite indefinidamente.

Métricas

Métricas en el enfoque tradicional (basado en procesos definidos)

Métricas de software

El dominio de las métricas se divide en tres:

- **De proceso:** miden **cómo trabaja la organización**, en todos sus proyectos. Por ejemplo, cuántos defectos se encuentran en promedio por proyecto, o qué tan precisas suelen ser las estimaciones. Sirven para **mejorar el proceso** a largo plazo.

- **De proyecto:** miden **un proyecto en particular**, mientras se ejecuta. Por ejemplo, cuánto esfuerzo se lleva gastado o si se cumple el cronograma. Sirven para **controlar el proyecto** y tomar decisiones.
- **De producto:** miden **el software en sí**: su tamaño, su complejidad o sus defectos. Sirven para evaluar su **calidad**.

Métricas básicas para un proyecto de software

Cuatro métricas básicas:

1. **Tamaño del producto:** qué tan grande es lo que se construye. Se puede medir en líneas de código, cantidad de requerimientos o casos de uso, puntos de función, etc.
2. **Esfuerzo:** cuánto **trabajo humano** se invierte, normalmente en **horas-persona**. Por ejemplo, 3 personas trabajando 10 horas son 30 horas-persona.
3. **Tiempo (calendario):** cuánto **tiempo transcurre** en el calendario, en días, semanas o meses. Es distinto del esfuerzo: 30 horas-persona pueden repartirse en 1 día (con mucha gente) o en 2 semanas (con una persona a tiempo parcial).
4. **Defectos:** cuántos defectos se encuentran, de qué tipo, en qué etapa y con qué severidad.

No hay que medir con más precisión ni más cantidad de lo que realmente se necesita. Antes de decidir medir algo, se propone tres preguntas:

- **¿Nos da más información que la que tenemos ahora?** Si no agrega nada nuevo, no vale la pena.
- **¿Es esta información de beneficio práctico?** ¿Sirve para tomar alguna decisión?
- **¿Nos dice lo que queremos saber?** ¿Responde la pregunta que tenemos?

Si la respuesta es "no", esa métrica es **desperdicio**: cuesta recolectarla y no se usa. Se conecta con el principio de calidad "simplicidad, empezar por lo básico".

Métricas en ambientes ágiles (para procesos empíricos)

"La medición es una salida, no una actividad." Las métricas deben **salir naturalmente del trabajo** que el equipo ya hace, y no ser una tarea extra que consuma tiempo. Por ejemplo, la velocidad sale de sumar los puntos de las historias terminadas, que ya están en el tablero. No hace falta que nadie se ponga a recolectar datos aparte.

Dos principios ágiles que guían la elección de las métricas

Son dos principios del **Manifiesto Ágil**:

- *"Nuestra mayor prioridad es satisfacer al cliente por medio de entregas tempranas y continuas de software valioso, funcionando."*
- *"El software funcionando es la principal medida de progreso."*

La consecuencia es que en ágil **no se mide el avance por documentos hechos ni por horas trabajadas**, sino por **software que funciona y se entrega**. Por eso las métricas ágiles miden lo que el equipo **terminó y entregó**.

Velocidad

La **velocidad** es la cantidad de **puntos de historia terminados** por el equipo en una iteración (sprint). Solo cuentan las historias **completamente terminadas**, que cumplen la definición de hecho. Una historia al 90% suma cero.

Cómo se usa:

- Para **planificar**: si el equipo hace en promedio unos 40 puntos por sprint, no conviene comprometer 70 en el próximo.
- Para **estimar cuándo se termina**: si quedan 400 puntos en el backlog y la velocidad es de 40, faltan unos 10 sprints.
- Una caída brusca, como la de la iteración 7, es una señal para preguntarse qué pasó: un feriado, gente enferma, un bloqueo o historias mal estimadas.

Un dato importante: la velocidad **es del equipo y no sirve para comparar equipos**, porque cada equipo estima sus puntos de forma distinta. Tampoco es una medida de productividad individual.

Capacidad

La **capacidad** es cuánto **puede trabajar el equipo** en una iteración, normalmente en **horas disponibles**. Depende de cuántas personas hay y cuánto tiempo pueden dedicar, descontando vacaciones, feriados, reuniones, etc.

Diferencia con la velocidad: la **capacidad** es lo que el equipo **puede** hacer (horas disponibles, se calcula **antes** del sprint para planificar). La **velocidad** es lo que el equipo **hizo** (puntos terminados, se mide **después** del sprint). Se usan juntas: si en el próximo sprint la capacidad baja a la mitad por vacaciones, hay que esperar una velocidad menor y comprometer menos trabajo.

Running Tested Features (RTF)

RTF significa "funcionalidades probadas y funcionando". Mide cuántas **funcionalidades están terminadas, pasan todas sus pruebas y funcionan** en el producto, a lo largo de las iteraciones.

Por qué es la métrica más fiel al principio ágil: mide exactamente **"software funcionando"**. Lo ideal es que la curva **suba siempre**. Si se estanca, el equipo no está entregando valor. Si baja, se está rompiendo lo que ya funcionaba, y eso se conecta con la **regresión automatizada**.

Métricas en Kanban (procesos empíricos con enfoque Lean)

Kanban: métricas clave

- **Lead Time = vista del cliente:** desde que el ítem **entra al backlog** hasta que está **completo**.
- **Cycle Time = vista interna:** desde que **se empieza a trabajar** el ítem (entra a Dev) hasta que está completo.

Cycle Time (tiempo de ciclo)

- Es el tiempo que pasa entre el **inicio y el final del proceso** para un ítem de trabajo dado. Se suele medir en días de trabajo o esfuerzo.
- Es una medición más **mecánica de la capacidad del proceso**: dice cuánto tarda el equipo en procesar un ítem una vez que lo empieza.
- Mide el **ritmo de terminación**.

Lead Time (tiempo de entrega)

- Es el tiempo que pasa entre el momento en que **se pide** un ítem de trabajo y el momento de **su entrega**. Se suele medir en días.
- Mide el **ritmo de entrega**.
- Es lo que **le importa al cliente**: cuánto tiene que esperar desde que pide algo hasta que lo recibe.

Ejemplo: un cliente pide una funcionalidad el día 1. Queda en el backlog esperando hasta el día 6, cuando el equipo la empieza. La terminan el día 10.

- **Lead time = 10 días** (del 1 al 10): lo que esperó el cliente.
- **Cycle time = 5 días** (del 6 al 10): lo que tardó el equipo en hacerla.

Los 5 días de diferencia son **espera en el backlog**. El cliente los sufre, aunque el equipo no los vea como "su" tiempo.

Touch Time (tiempo de tocado)

- Es el tiempo en que un ítem fue **realmente trabajado** ("tocado") por el equipo.
- Se cuenta cuántos días hábiles pasó el ítem en columnas de **"trabajo en curso"**, a diferencia de las columnas de **cola o buffer** (como "Hecho" o "Listo para..."), y del tiempo en que estuvo **bloqueado** o sin que nadie trabajara sobre él.

Siguiendo el ejemplo: de esos 5 días de cycle time, quizás solo 2 alguien estuvo trabajando en el ítem, y los otros 3 estuvo esperando entre etapas (por ejemplo, terminado en Dev pero esperando a que Test lo tome). **Touch time = 2 días.**

Métricas orientadas a servicio

Como Kanban ve el trabajo como un **servicio** al cliente, define cuatro **niveles de servicio**:

- **Expectativa del nivel de servicio:** lo que **los clientes esperan**. Por ejemplo, "esperamos que nuestros pedidos se entreguen en una semana".
- **Capacidad del nivel de servicio:** lo que **el sistema realmente puede entregar**, según las métricas. Por ejemplo, "medimos que entregamos el 85% de los pedidos en 8 días".
- **Acuerdo de nivel de servicio:** lo que **se acuerda formalmente con el cliente**. Por ejemplo, "nos comprometemos a entregar el 80% de los pedidos en 10 días".

- **Umbral de adecuación del servicio:** el nivel **por debajo del cual el servicio es inaceptable** para el cliente. Por ejemplo, "si tardan más de 20 días, el servicio no nos sirve".

Resumiendo métricas en cada enfoque

Tradicionales	Ágiles	Lean (Kanban)
Esfuerzo	Velocidad	Lead Time
Tiempo	Capacidad	Cycle Time
Costos	Running Tested Features	Touch Time
Defectos		Eficiencia del proceso

La lógica de cada columna:

- **Tradicionales:** miden el **cumplimiento del plan**: si se gasta el esfuerzo, el tiempo y el costo planificados, y cuántos defectos hay.
- **Ágiles:** miden el **valor entregado** en cada iteración: cuánto se terminó, con qué capacidad, y cuánto software funciona.
- **Lean:** miden el **flujo**: cuánto tarda el trabajo en atravesar el sistema y cuánto de ese tiempo es espera.

¿Y para el producto software, qué métricas?

Más allá del enfoque de gestión, el **producto** se mide en dos dimensiones:

Tamaño

- **Líneas de código.**
- **Requerimientos**, que a su vez se pueden contar como **casos de uso**, **features**, **clases** o **puntos de historia**.

Defectos

- **Cobertura:** qué porcentaje del código se ejecuta con las pruebas. Es la cobertura de caja blanca que vimos en testing.
- **Defectos por severidad:** cuántos bloqueantes, críticos, mayores, menores y cosméticos.
- **Densidad de defectos:** cantidad de defectos en relación con el tamaño. Por ejemplo, defectos cada mil líneas de código, o por caso de uso. Permite comparar productos de distinto tamaño: 50 defectos en un sistema enorme no es lo mismo que 50 en uno chico.

¿Cuál es mejor?

Ninguno es "mejor" en general: cada uno sirve para algo distinto. Con los enfoques y sus métricas pasa lo mismo. No hay un enfoque ni un conjunto de métricas superior. Hay que elegir según el **contexto**: el tipo de proyecto, el equipo y lo que se necesita saber.

Auditorías

Evaluación **independiente** de los productos o procesos de software para asegurar el **cumplimiento con estándares, lineamientos, especificaciones y procedimientos**, basada en un **criterio objetivo**, incluyendo documentación que especifique:

1. La forma o el contenido de los productos a desarrollar.
2. El proceso por el cual se desarrollan los productos.
3. Cómo debería medirse el cumplimiento de los estándares o lineamientos.

Características: son **objetivas e independientes**. Quien audita no participó del trabajo, y se compara contra algo escrito (estándares, procedimientos, plan), no contra opiniones.

Diferencia con una revisión técnica

La revisión técnica la hacen **pares técnicos** y busca **defectos en el contenido** del producto. La auditoría la hace **alguien independiente** (normalmente el grupo de calidad) y busca **evidencia de que se cumplió el proceso**. Por ejemplo, una revisión encuentra que un diseño tiene un error de lógica. Una auditoría verifica que el diseño **haya pasado por la revisión** que exige el proceso.

Tipos

Tipo	Sobre qué	Qué valida
De proyecto	**Proceso**	Que se cumpla el proceso de desarrollo definido
De configuración funcional	**Producto**	Que el software cumpla con sus **requerimientos** (código vs. requerimientos)
De configuración física	**Producto**	Que el software sea consistente con su **documentación** (código vs. documentación)

Los pasos de la auditoría:

1. **Planificación:** acordar fecha, alcance, objetivo y participantes; preparar la documentación.
2. **Ejecución:** reunión de apertura; después preguntas, pedido de evidencia y checklist.
3. **Análisis y reporte:** evaluar los resultados, reunión de cierre y entrega del reporte final.
4. **Seguimiento:** el auditado propone un plan de acción; el auditor lo aprueba, verifica que se cumpla y cierra las desviaciones.

Hallazgos

Hallazgo	Qué es	¿Requiere plan de acción?
No conformidad (en la filmina se llama **desviación**)	Un incumplimiento concreto del proceso o de los requerimientos	**Sí**
Observación	Algo mejorable, o un riesgo de desviación a futuro	No
Buena práctica	Algo hecho mucho mejor de lo esperado	No

Auditoría al grupo de calidad.

Es una auditoría **al área que hace las auditorías**, para **revisar que se esté auditando y que se cumpla el plan de calidad**. Responde a la pregunta "¿quién controla al que controla?".

Checklist de auditoría

El checklist es la guía que usa el auditor. Su contenido general:

- Fecha de la auditoría.
- Lista de auditados, con su rol.
- Nombre del auditor.
- Nombre del proyecto.
- Fase actual del proyecto, si corresponde.
- Objetivo y alcance de la auditoría.
- **Lista de preguntas.**

Revisiones técnicas

Objetivo

Buscan la **detección temprana de defectos**, para **asegurar la calidad del producto**. Como no necesitan que el programa se ejecute, se pueden aplicar desde los requerimientos.

Verificación y Validación

- **Validación** relaciona el mundo real con la definición del problema. Busca **correspondencia**: ¿lo que definimos es lo que el mundo real necesita?
- **Verificación** relaciona la definición del problema con la implementación. Busca **corrección**: ¿lo que construimos respeta lo que definimos?

Falla y producto de trabajo

- **Falla:** un error en un producto de trabajo.
- **Producto de trabajo:** la salida de cualquier actividad del ciclo de vida: un documento de requerimientos, un diseño, código, un plan de prueba.

Ventajas

- **Pueden descubrirse muchos errores** en una sola revisión. En testing, en cambio, un defecto puede ocultar a otros.
- **Pueden inspeccionarse versiones incompletas**, porque no hace falta que el sistema funcione.
- **Pueden considerarse otros atributos de calidad**, como el cumplimiento de estándares o la mantenibilidad, que las pruebas no ven.

Desventajas

- **Es difícil introducir las inspecciones formales** en una organización.
- **Sobrecargan los costos al inicio**, y el ahorro llega solo cuando los equipos ganan experiencia.
- **Requieren tiempo para organizarse** y **parecen hacer más lento** el desarrollo.

Tipos

	Formales	**Informales**
Ejemplo	**Inspecciones de software**	**Walkthrough** (recorrido)
Proceso	Formal, con roles, checklists y métricas	Informal, sin preparación ni métricas
Quién conduce	El moderador	El propio autor

Inspección

Es una **actividad de garantía de calidad** de software. Sus objetivos:

- **Descubrir errores.**
- **Verificar** que el software cumple sus requisitos.
- **Garantizar** que el software está representado según ciertos **estándares**.
- Conseguir un software desarrollado de forma **uniforme**.
- Hacer que los proyectos sean **más manejables**.

Se lleva a cabo mediante una **reunión**, y su éxito depende de la **planificación**.

Etapa	Qué se hace
1. Planificación	Se elige el equipo y se preparan el material y el calendario
2. Visión general	Se presentan el proceso y el producto a los participantes
3. Preparación	Cada inspector analiza el producto **por su cuenta** para encontrar posibles defectos
4. Reunión de inspección	El equipo junta los posibles defectos encontrados, y **filtra los falsos positivos** (cosas que parecían defectos y no lo son)
5. Corrección	El autor corrige los defectos
6. Seguimiento	Se verifican las correcciones y se recolectan los datos

Al final de la reunión se toma una decisión:

- **Aceptar** el producto sin modificaciones.
- **Rechazar** el producto: tiene errores graves, y después de corregirlos hay que volver a inspeccionarlo.
- **Aceptarlo provisoriamente:** tiene errores menores que hay que corregir, pero no hace falta otra inspección.

Walkthrough (recorrida)

Es una **técnica de análisis estático** en la que un **diseñador o programador guía** a miembros del equipo y otros interesados **a través de un producto** de software. Los participantes hacen **preguntas y comentarios** sobre posibles errores, violaciones de estándares y otros problemas.

La diferencia con la inspección es que acá **lo conduce el propio autor**, y el proceso es informal: no hay roles fijos, ni preparación obligatoria, ni métricas.